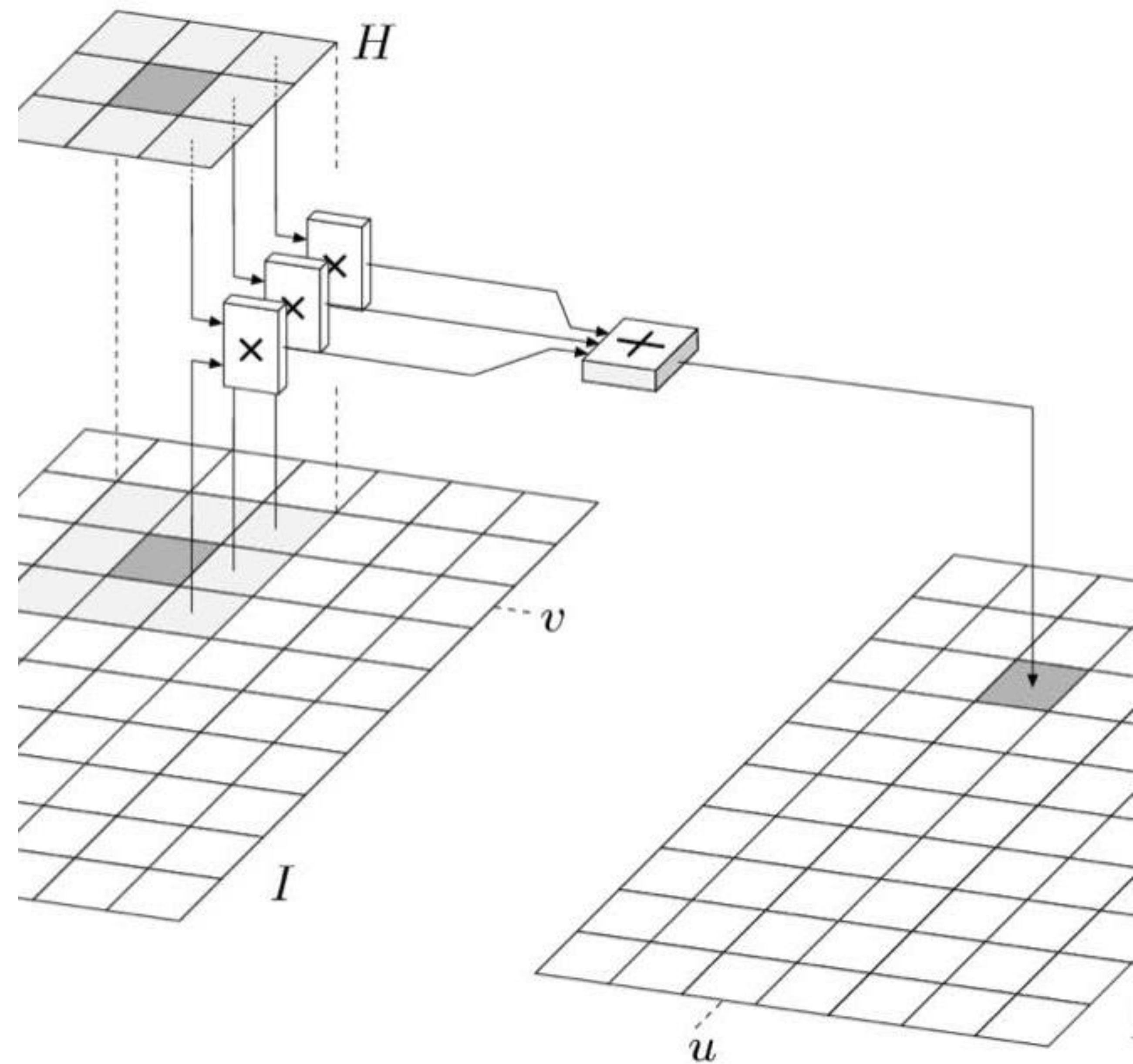


Computer Vision & AI

SW 05

Linear Filters
Convolution
Correlation

14. März 2024



Why?

Why filters?

- extraction of details
 - edges
 - corners
- noise removal
- image modification in general

Why correlation?

- pattern matching
- pattern localization

1D Convolution (Definition)

Given two sequences

$$x : \mathbb{Z} \rightarrow \mathbb{R}$$
$$i \mapsto x_i$$

$$y : \mathbb{Z} \rightarrow \mathbb{R}$$
$$i \mapsto y_i$$

Discrete convolution of x and y:

$$\text{conv}(x, y)_k = \sum_{i=-\infty}^{\infty} x_i y_{k-i} = \sum_{i=-\infty}^{\infty} x_{k-i} y_i$$

Example:

$$x_k = \begin{cases} 1 & \text{if } 0 \leq k < 4, \\ 0 & \text{otherwise.} \end{cases}$$

$$y_k = \begin{cases} 1 & \text{if } k \in \{0, 2\}, \\ 2 & \text{if } k = 1, \\ 0 & \text{otherwise.} \end{cases}$$

Computation:

k	-4	-3	-2	-1	0	1	2	3	4	5	6
x_k	0	0	0	0	1	1	1	1	0	0	0
y_k	0	0	0	0	1	2	1	0	0	0	0

$$z_k = \text{conv}(x, y)_k$$

$$z_{-1} = x_0 y_{-1} + x_1 y_{-2} + x_2 y_{-3} + x_3 y_{-4} = 1 \cdot 0 + 1 \cdot 0 + 0 \cdot 1 + 1 \cdot 0 = 0$$

$$z_0 = x_0 y_0 + x_1 y_{-1} + x_2 y_{-2} + x_3 y_{-3} = 1 \cdot 1 + 1 \cdot 0 + 0 \cdot 1 + 1 \cdot 0 = 1$$

$$z_1 = x_0 y_1 + x_1 y_0 + x_2 y_{-1} + x_3 y_{-2} = 1 \cdot 2 + 1 \cdot 1 + 0 \cdot 1 + 1 \cdot 0 = 3$$

$$z_2 = x_0 y_2 + x_1 y_1 + x_2 y_0 + x_3 y_{-1} = 1 \cdot 1 + 1 \cdot 2 + 1 \cdot 1 + 1 \cdot 0 = 4$$

$$z_3 = x_0 y_3 + x_1 y_2 + x_2 y_1 + x_3 y_0 = 1 \cdot 0 + 1 \cdot 1 + 2 \cdot 1 + 1 \cdot 1 = 4$$

$$z_4 = x_0 y_4 + x_1 y_3 + x_2 y_2 + x_3 y_1 = 1 \cdot 0 + 1 \cdot 0 + 1 \cdot 1 + 1 \cdot 2 = 3$$

$$z_5 = x_0 y_5 + x_1 y_4 + x_2 y_3 + x_3 y_2 = 1 \cdot 0 + 1 \cdot 0 + 0 \cdot 1 + 1 \cdot 1 = 1$$

$$z_6 = x_0 y_6 + x_1 y_5 + x_2 y_4 + x_3 y_3 = 1 \cdot 0 + 1 \cdot 0 + 0 \cdot 1 + 1 \cdot 0 = 0$$

$$z_k = 0 \text{ for } k \leq -1 \text{ or } k \geq 6.$$

1D Convolution (Example)

i	-3	-2	-1	0	1	2	3	4	5	6	
x_i	0	0	0	1	1	1	1	0	0	0	first sequence
y_i	0	0	0	1	2	1	0	0	0	0	second sequence
y_{0-i}	0	1	2	1	0	0	0	0	0	0	reflected ...
y_{1-i}	0	0	1	2	1	0	0	0	0	0	right shifted ...
y_{2-i}	0	0	0	1	2	1	0	0	0	0	...
y_{3-i}	0	0	0	0	1	2	1	0	0	0	...
y_{4-i}	0	0	0	0	0	1	2	1	0	0	...
y_{5-i}	0	0	0	0	0	0	1	2	1	0	
y_{6-i}	0	0	0	0	0	0	0	1	2	1	...

$z_k = \text{conv}(x, y)_k$

$z_k = 0 \text{ for } k < 0.$

$z_0 = 1 \cdot 1 = 1$

$z_1 = 1 \cdot 2 + 1 \cdot 1 = 3$

$z_2 = 1 \cdot 1 + 1 \cdot 2 + 1 \cdot 1 = 4$

$z_3 = 1 \cdot 1 + 1 \cdot 2 + 1 \cdot 1 = 4$

$z_4 = 1 \cdot 1 + 1 \cdot 2 = 3$

$z_5 = 1 \cdot 1 = 1$

$z_k = 0 \text{ for } k > 5.$

1D Convolution (Properties)

The convolution is

- associative: $\text{conv}(\text{conv}(x, y), z) = \text{conv}(x, \text{conv}(y, z))$
- commutative: $\text{conv}(x, y) = \text{conv}(y, x)$
- distributive: $\text{conv}(x + y, z) = \text{conv}(x, z) + \text{conv}(y, z)$

1D Convolution (Delta Sequence)

Definition

$$\delta_k = \begin{cases} 1 & \text{if } k = 0, \\ 0 & \text{otherwise.} \end{cases}$$

Convolution with delta sequence

$$\text{conv}(\delta, x)_k = \sum_{i=-\infty}^{\infty} \delta_i x_{k-i} = 1 \cdot x_k = x_k$$

Shifted delta sequence

$$\delta_k^i = \delta_{k-i} = \begin{cases} 1 & \text{if } k = i, \\ 0 & \text{otherwise.} \end{cases}$$

Example:

Given the sequence

$$x = (1, 3, 4, 2),$$

compute the convolution

$$\text{conv}(x, \delta^0 + \delta^6 + \delta^{10}).$$

2D Convolution

$$I'(u, v) = \sum_{(i,j) \in R} I(u - i, v - j) \cdot \tilde{H}(i, j)$$

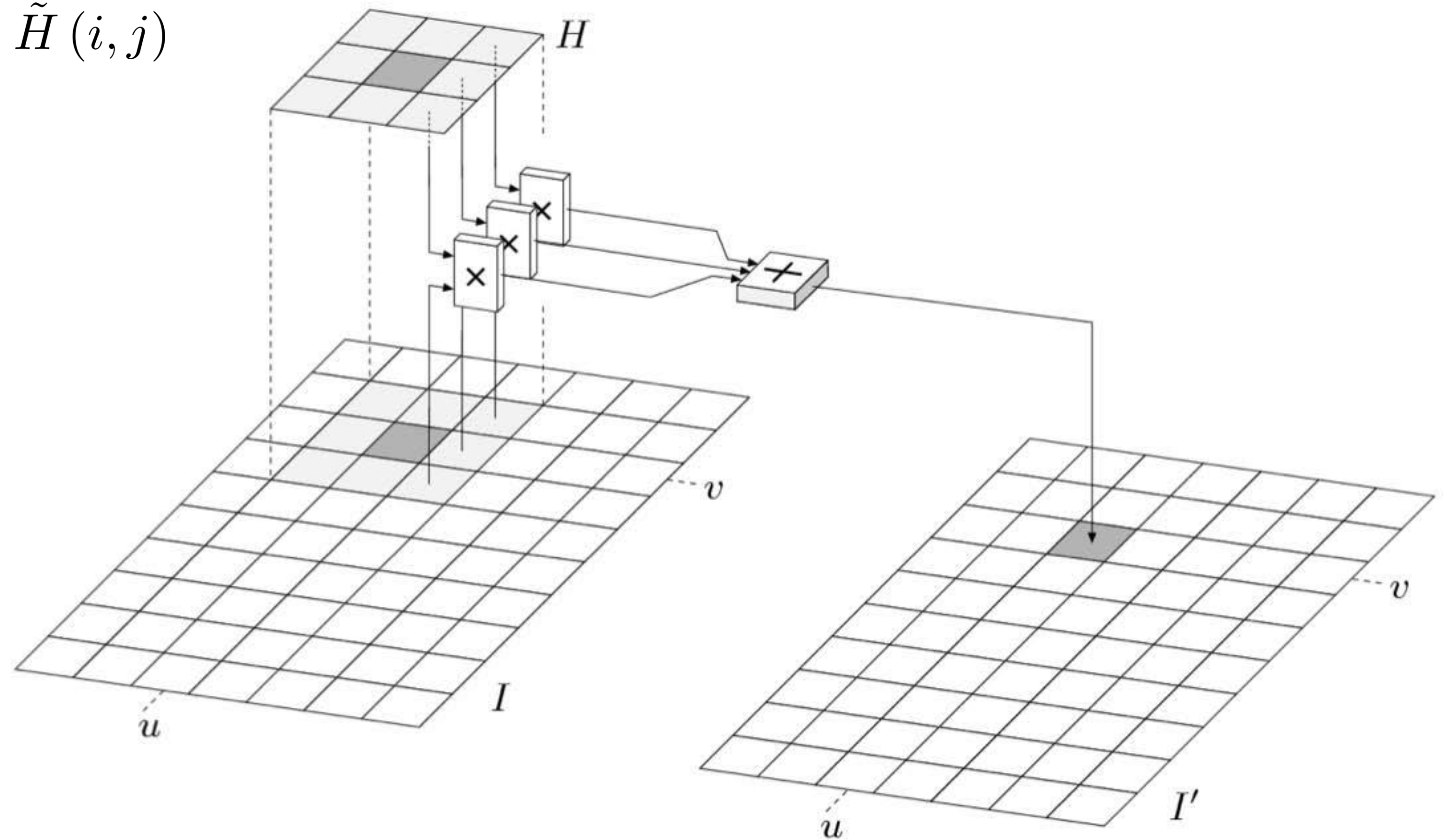
I original image

$\tilde{H}$ filter matrix (filter kernel)

H filter matrix, rotated by 180°

R range of $\tilde{H}$'s indices

I' convolution: $\text{conv}(I, \tilde{H})$



2D Convolution (continued)

$$\begin{aligned} I'(u, v) &= \sum_{i=-m}^m \sum_{j=-n}^n I(u-i, v-j) \cdot \tilde{H}(i, j) \\ &= \sum_{i=-m}^m \sum_{j=-n}^n I(u+i, v+j) \cdot \tilde{H}(-i, -j) \\ &= \sum_{i=-m}^m \sum_{j=-n}^n I(u+i, v+j) \cdot H(i, j) \end{aligned}$$

Properties

- $I * \tilde{H}$ is linear in $\tilde{H}$ as well as in H .
- $\tilde{H}$ is centered around $\tilde{H}(0, 0)$.
- $\tilde{H}$ has $2m + 1$ lines and $2n + 1$ columns.
- wide-used parameters: $m = n = 1$:
 $\tilde{H}$ has 3 lines and 3 columns.

2D Convolution (continued)



original image

Low-pass filter

$$\tilde{H} = \frac{1}{9} \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$$

$$I'(u, v) = \frac{1}{9} \sum_{i=-1}^1 \sum_{j=-1}^1 I(u+i, v+j) \cdot \tilde{H}(-i, -j)$$

$$= \frac{1}{9} \sum_{i=-1}^1 \sum_{j=-1}^1 I(u+i, v+j)$$

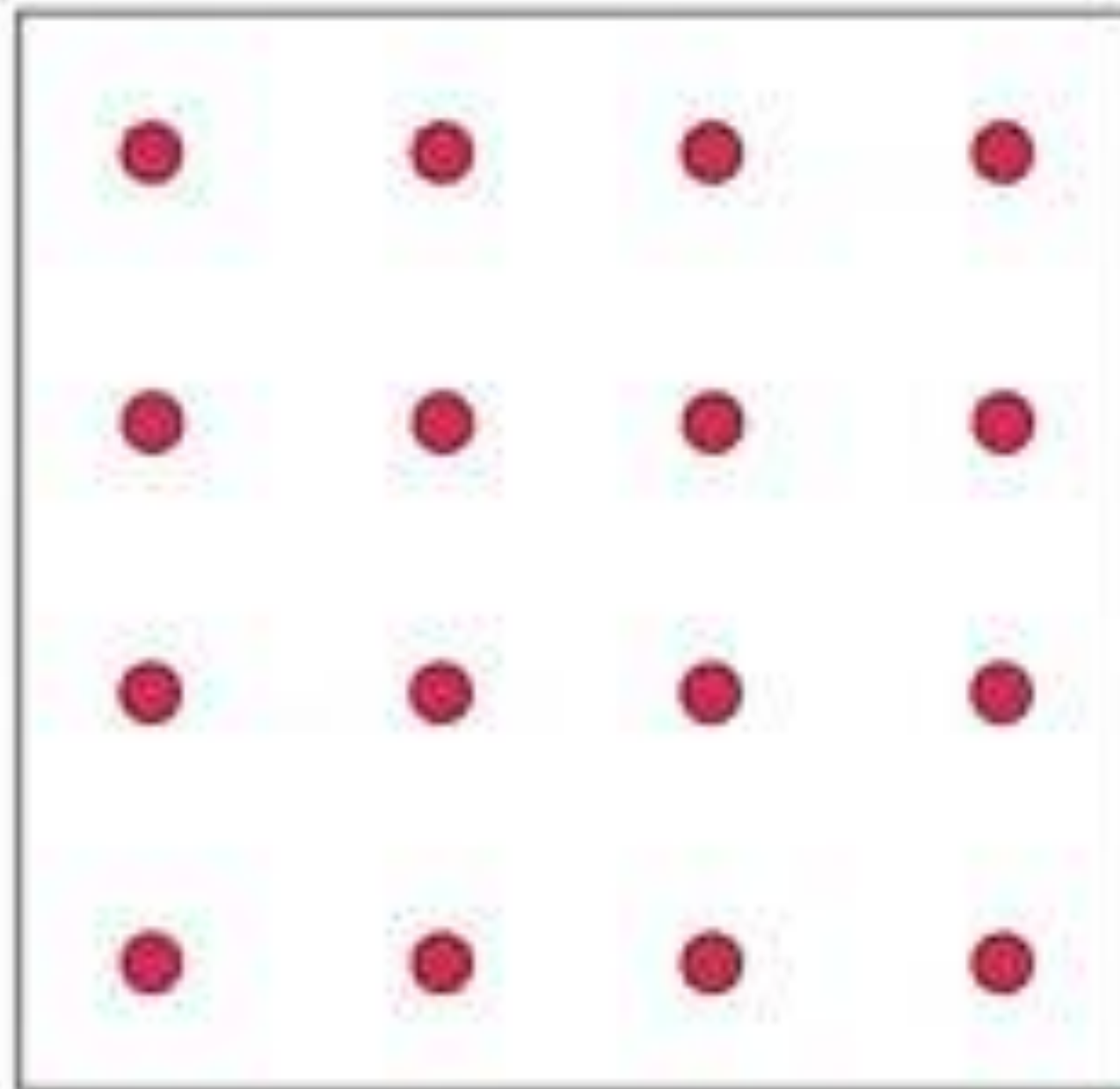
filtered (smoothed) image



2D Convolution (continued)

placing objects

Convolution

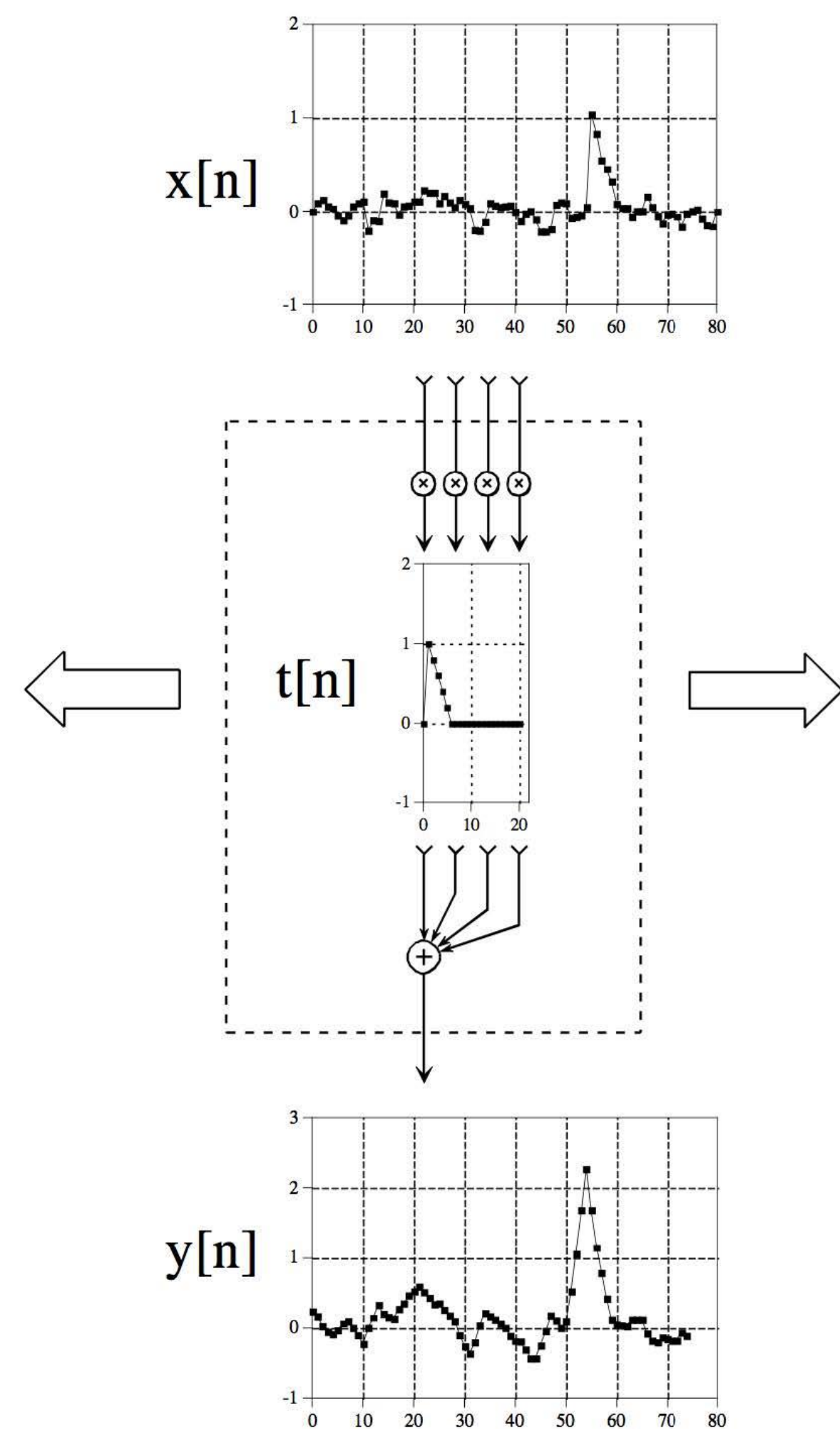
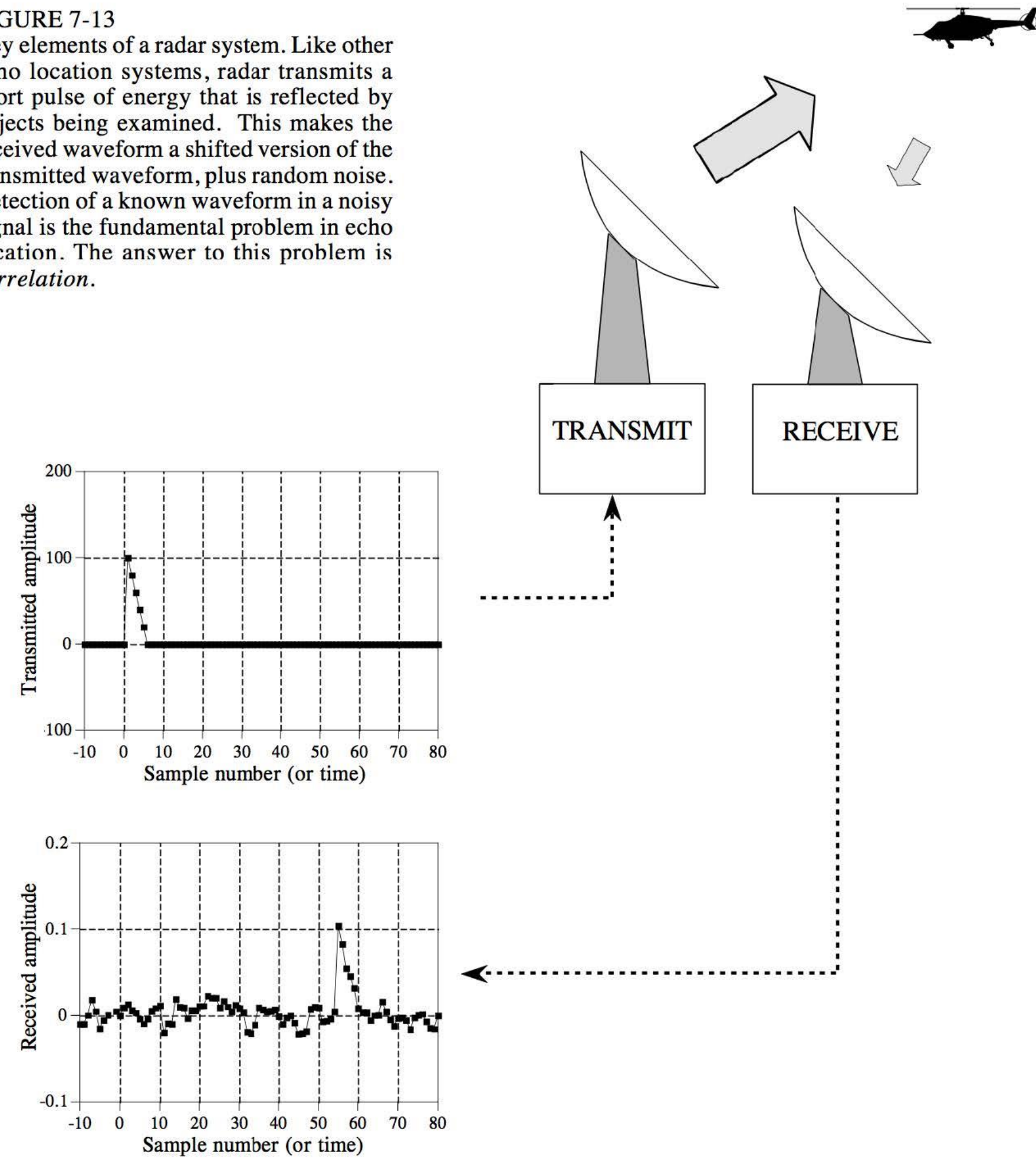


=



1D (Cross-) Correlation

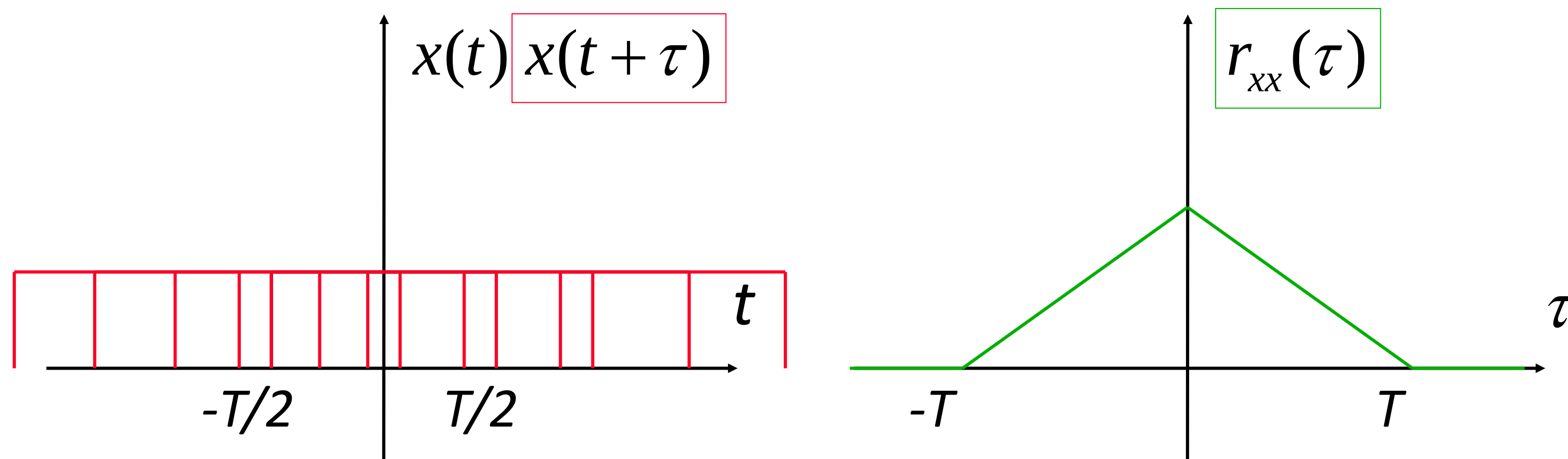
FIGURE 7-13
Key elements of a radar system. Like other echo location systems, radar transmits a short pulse of energy that is reflected by objects being examined. This makes the received waveform a shifted version of the transmitted waveform, plus random noise. Detection of a known waveform in a noisy signal is the fundamental problem in echo location. The answer to this problem is *correlation*.



Autocorrelation Function

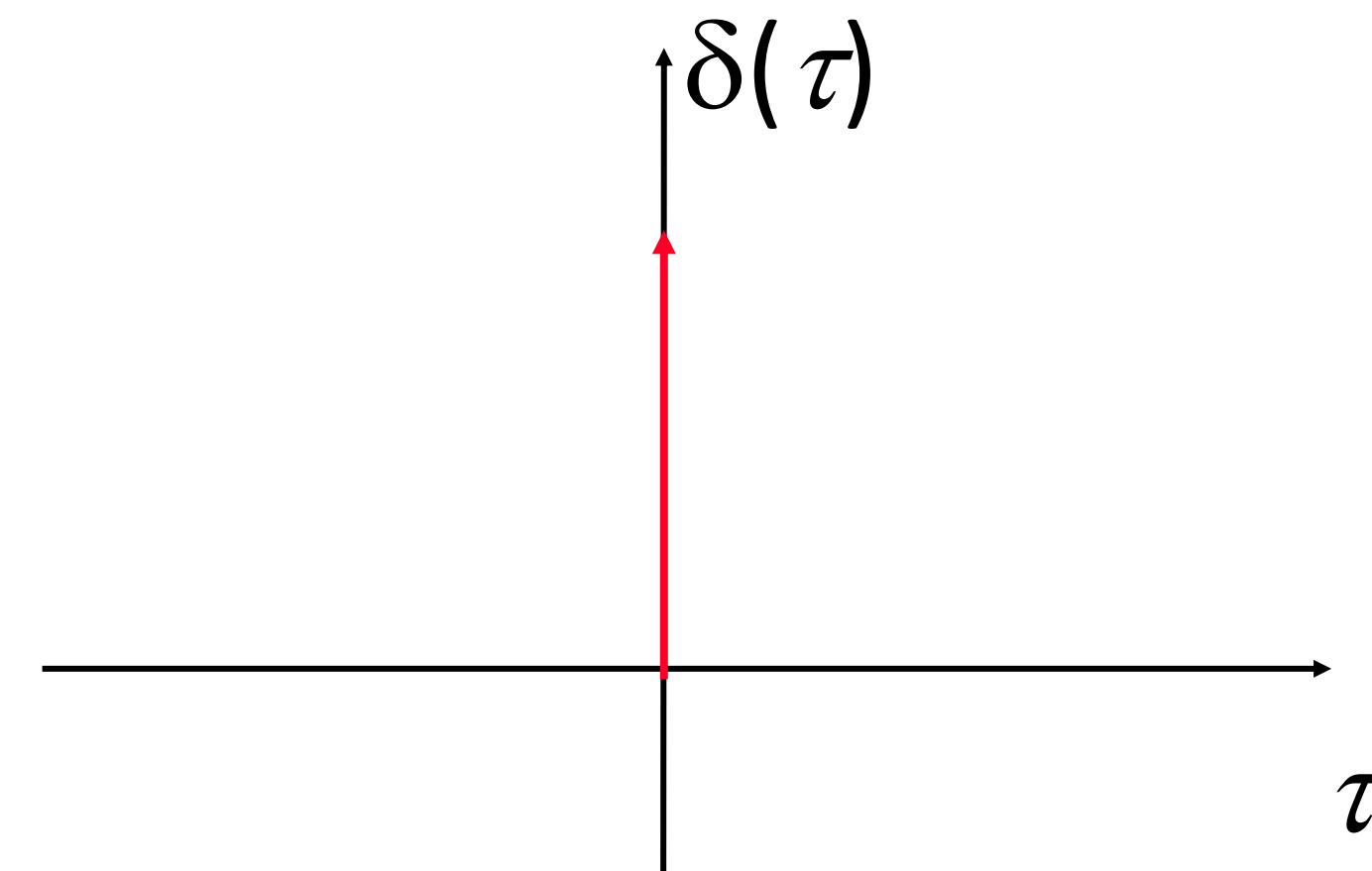
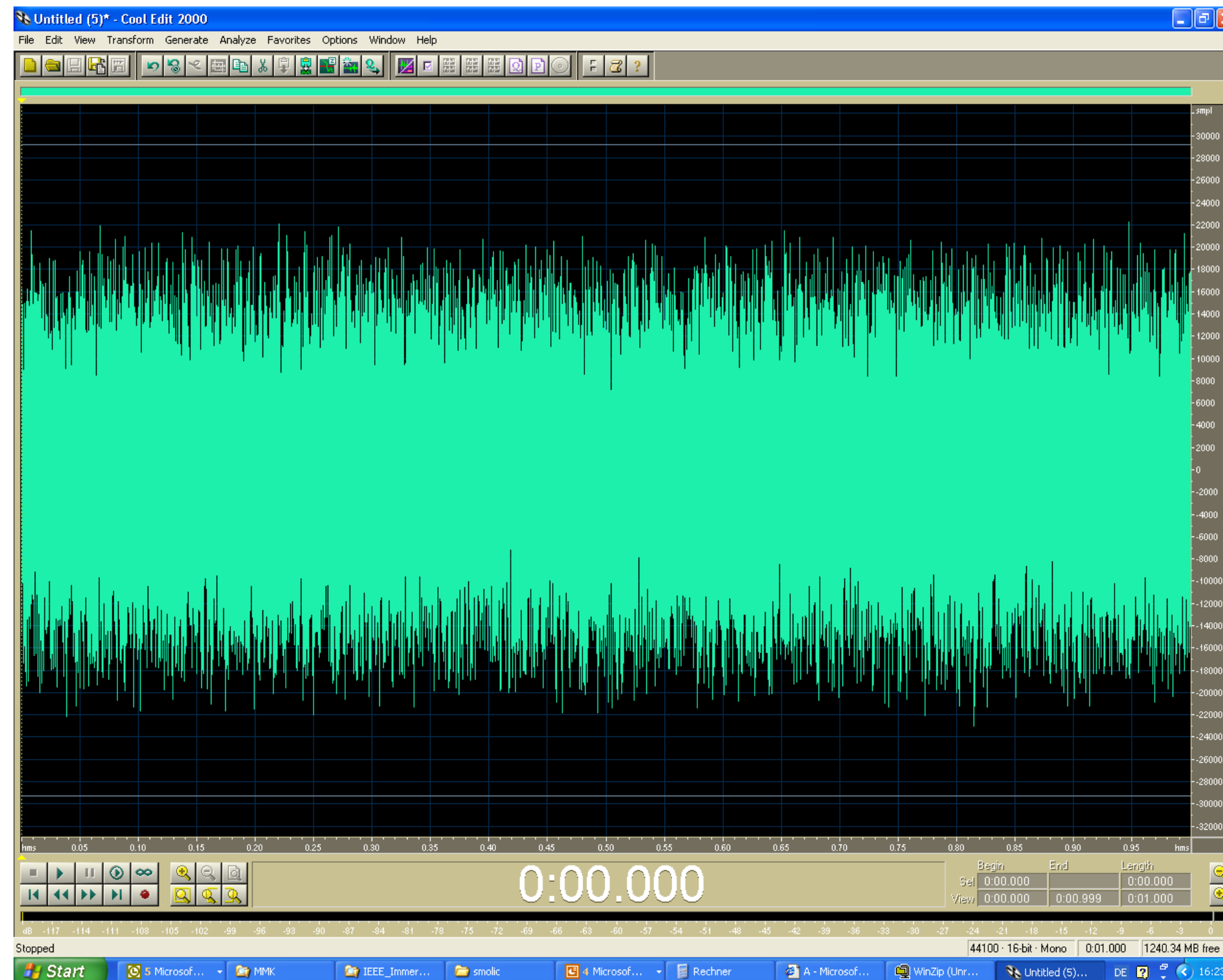
- Autocorrelation function (ACF): Measure for the similarity of a signal with itself (with the time shifted version)

$$r_{xx}(\tau) = \int_{-\infty}^{\infty} x(t) \cdot x(t + \tau) dt$$



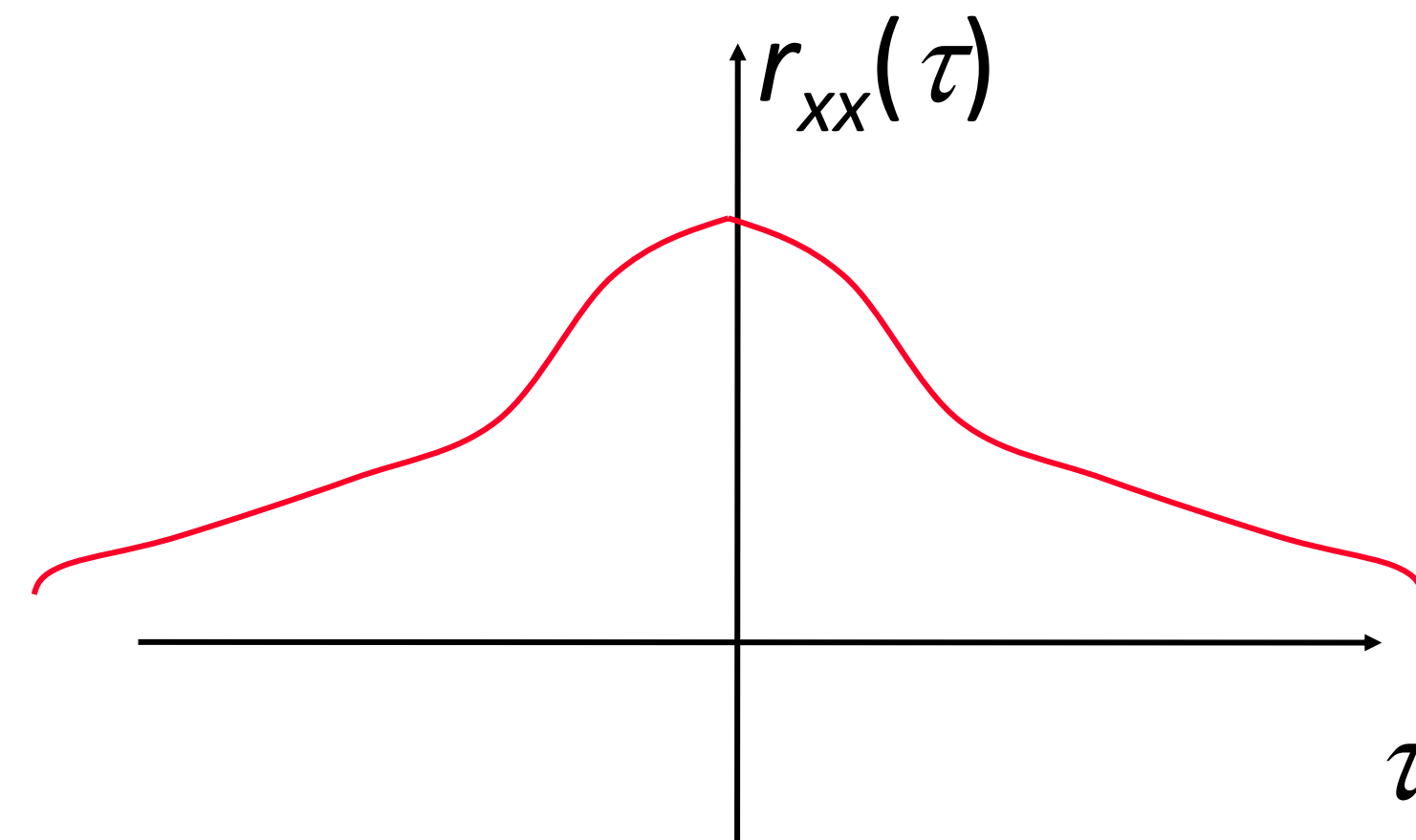
ACF of White Noise

- ACF of noise is a delta function, complete elimination besides at $\tau = 0$



Autocorrelation Function

- All other ACFs have a certain shape



Autocorrelation Function

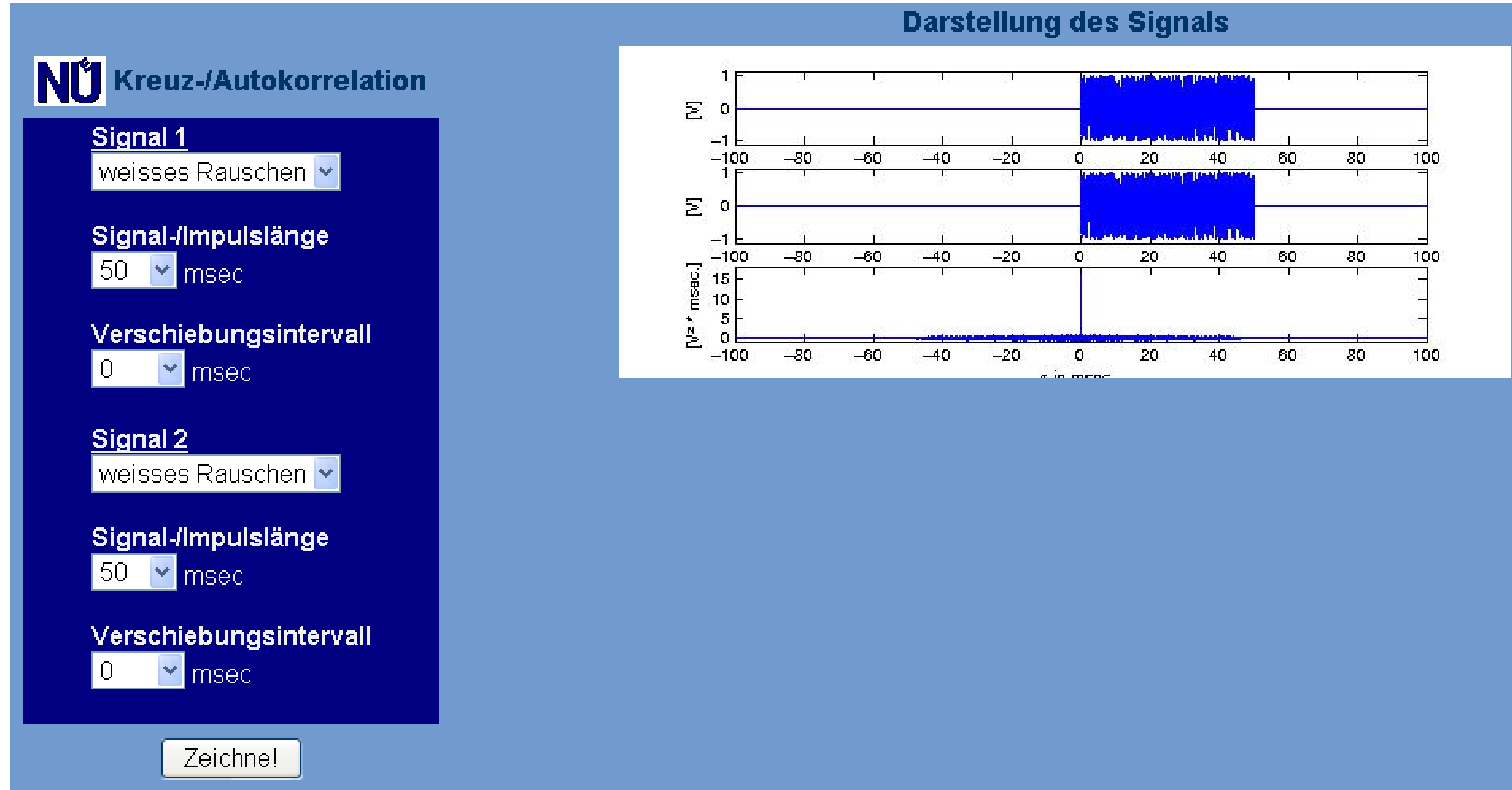
- Discrete ACF:

$$r_{xx}(k) = \sum_{n=-\infty}^{\infty} x(n) \cdot x(n+k)$$

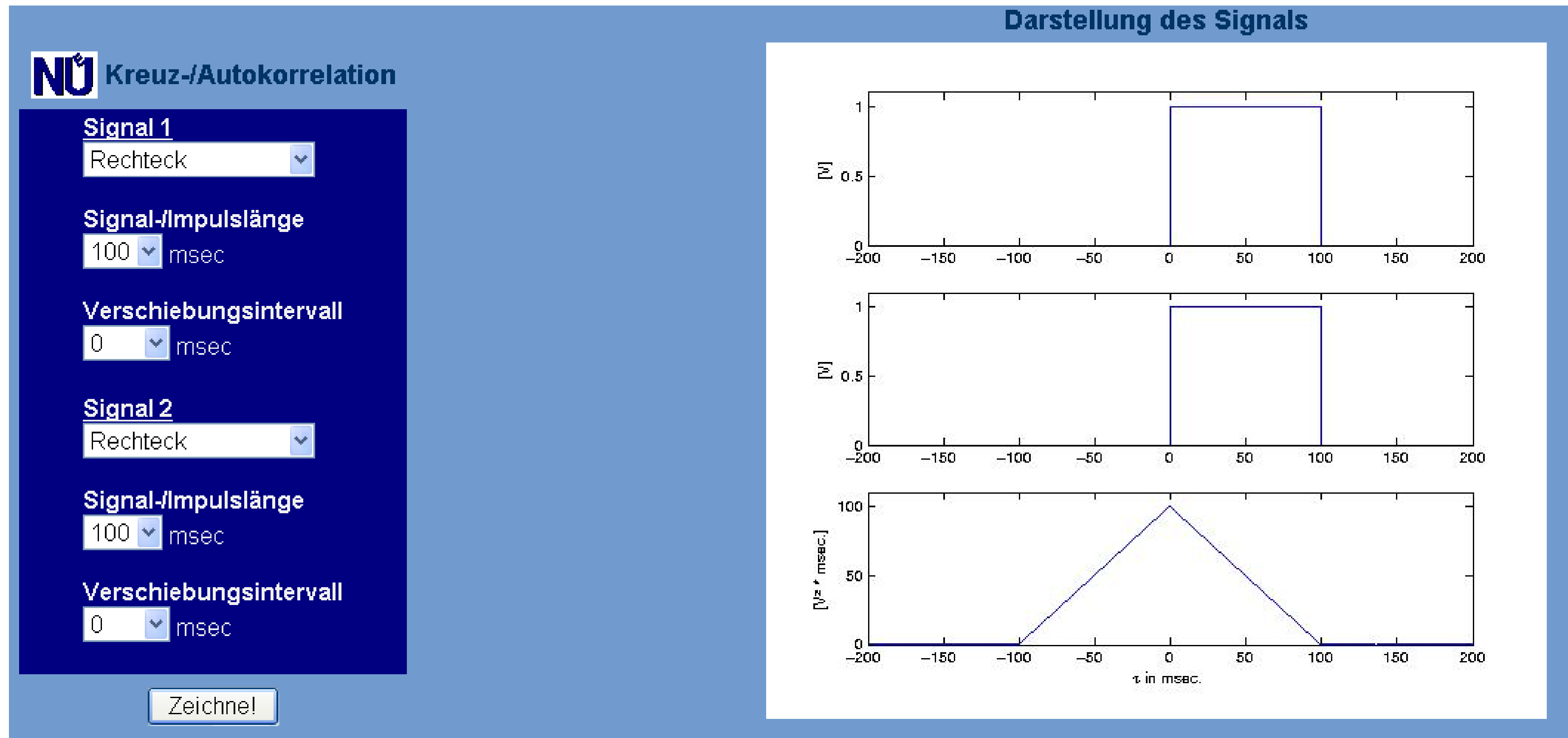
- Normalized ACF, autocorrelation coefficient

$$\rho_{xx}(k) = \frac{r_{xx}(k)}{r_{xx}(0)}$$

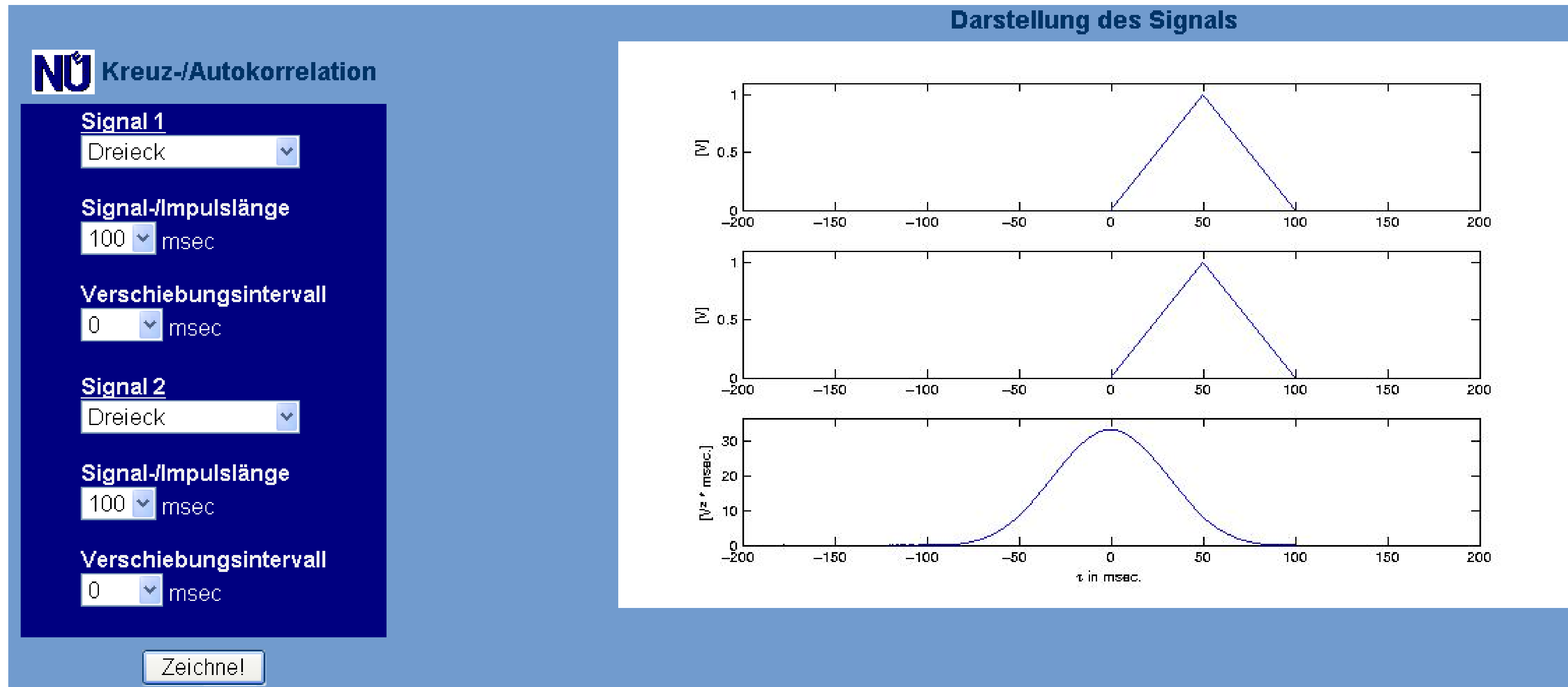
ACF of Noise



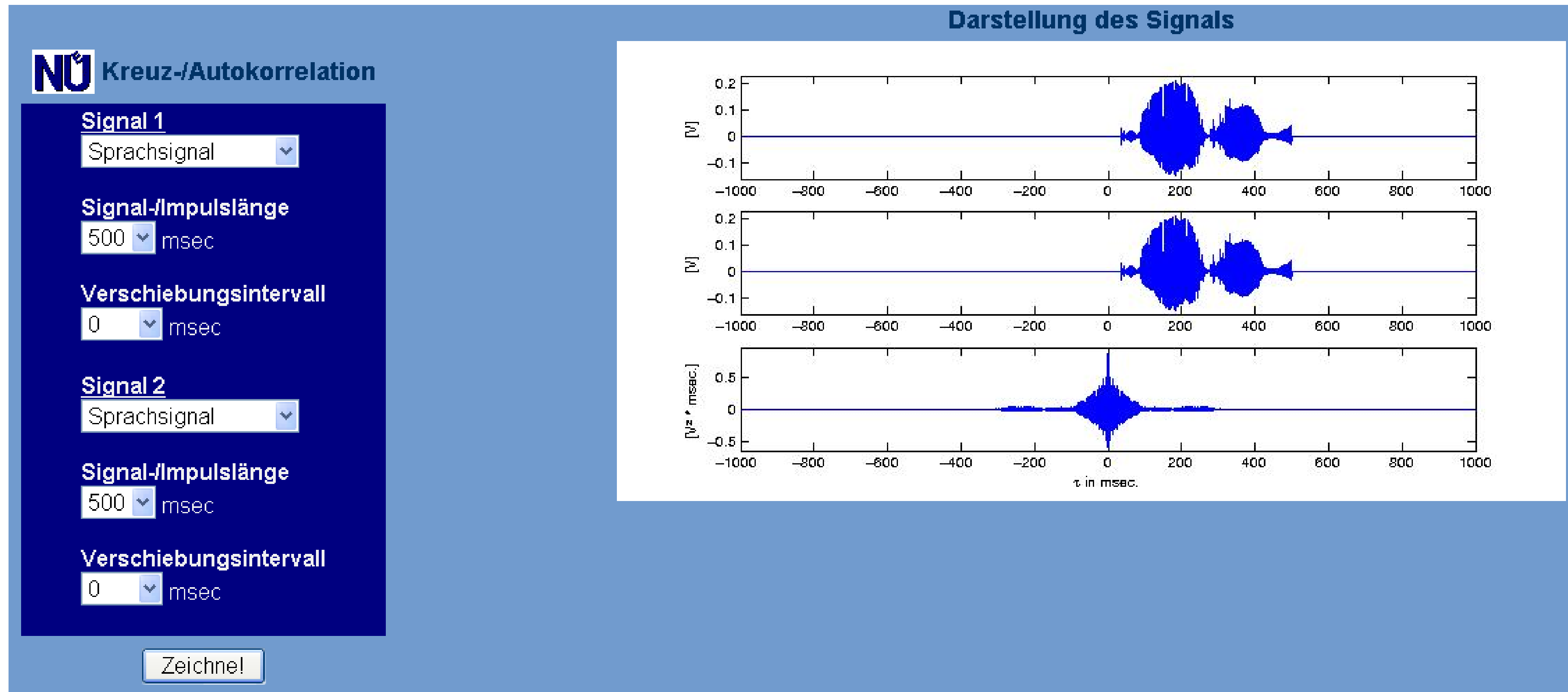
ACF of a Rectangular Function



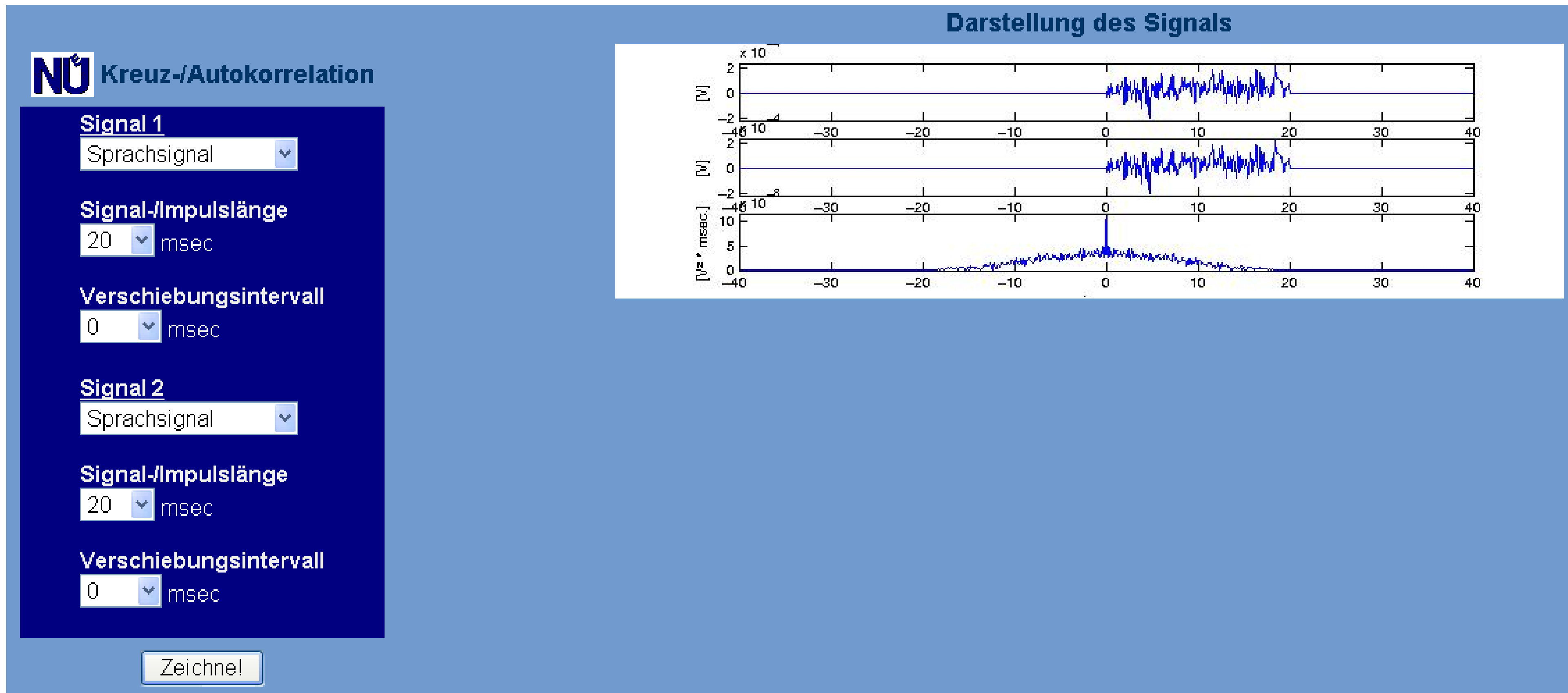
ACF of a Triangular Function



ACF of a Speech Signal

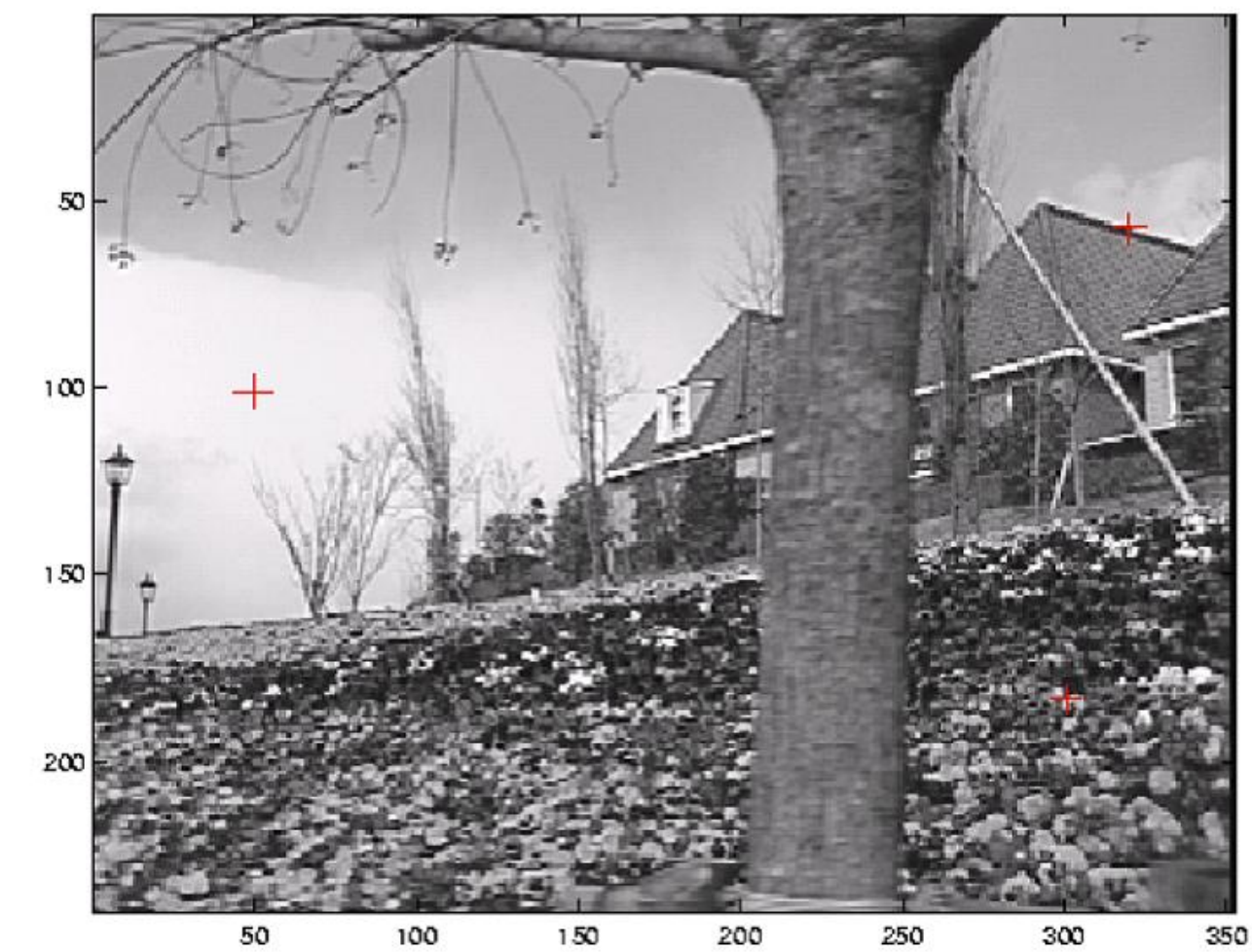


ACF of a Speech Signal



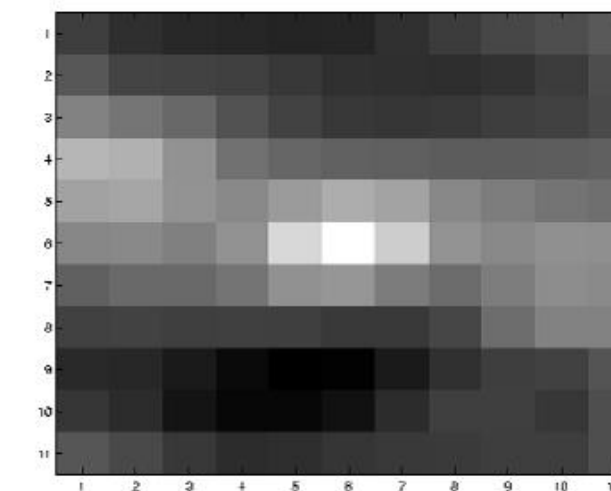
Feature extraction

$$E_{AC}(\Delta \mathbf{u}) = \sum_i w(\mathbf{x}_i) [I_0(\mathbf{x}_i + \Delta \mathbf{u}) - I_0(\mathbf{x}_i)]^2$$

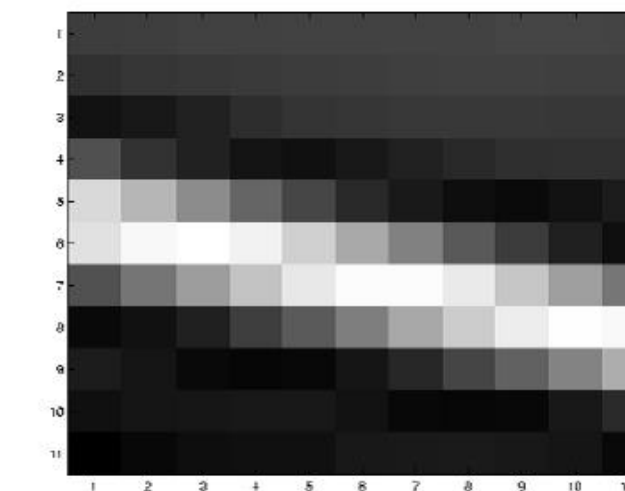


(a)

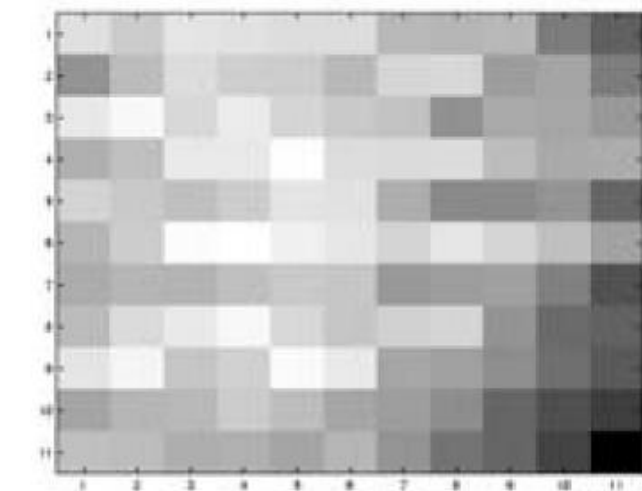
- Similarity measure (AC), a good test for suitability of a point as feature



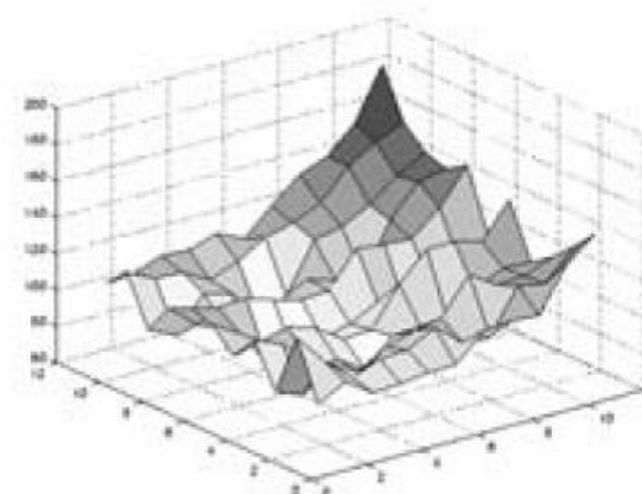
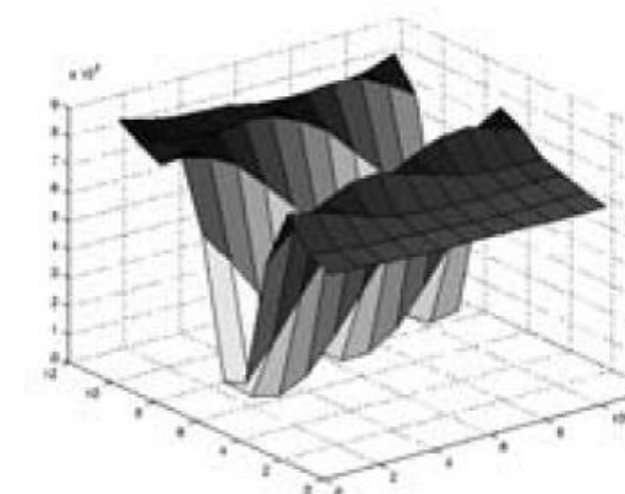
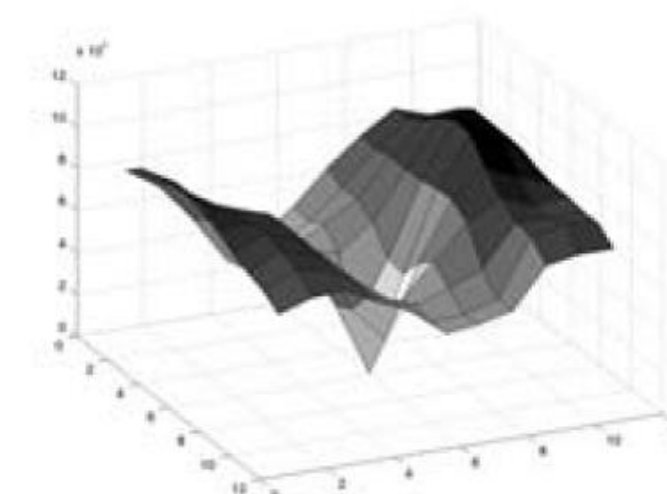
(b)



(c)



(d)



▪ http://szeliski.org/Book/drafts/SzeliskiBook_20100903_draft.pdf

▪ Computer Vision: Algorithms and Applications, Richard Szeliski, September 3, 2010 draft, 2010 Springer

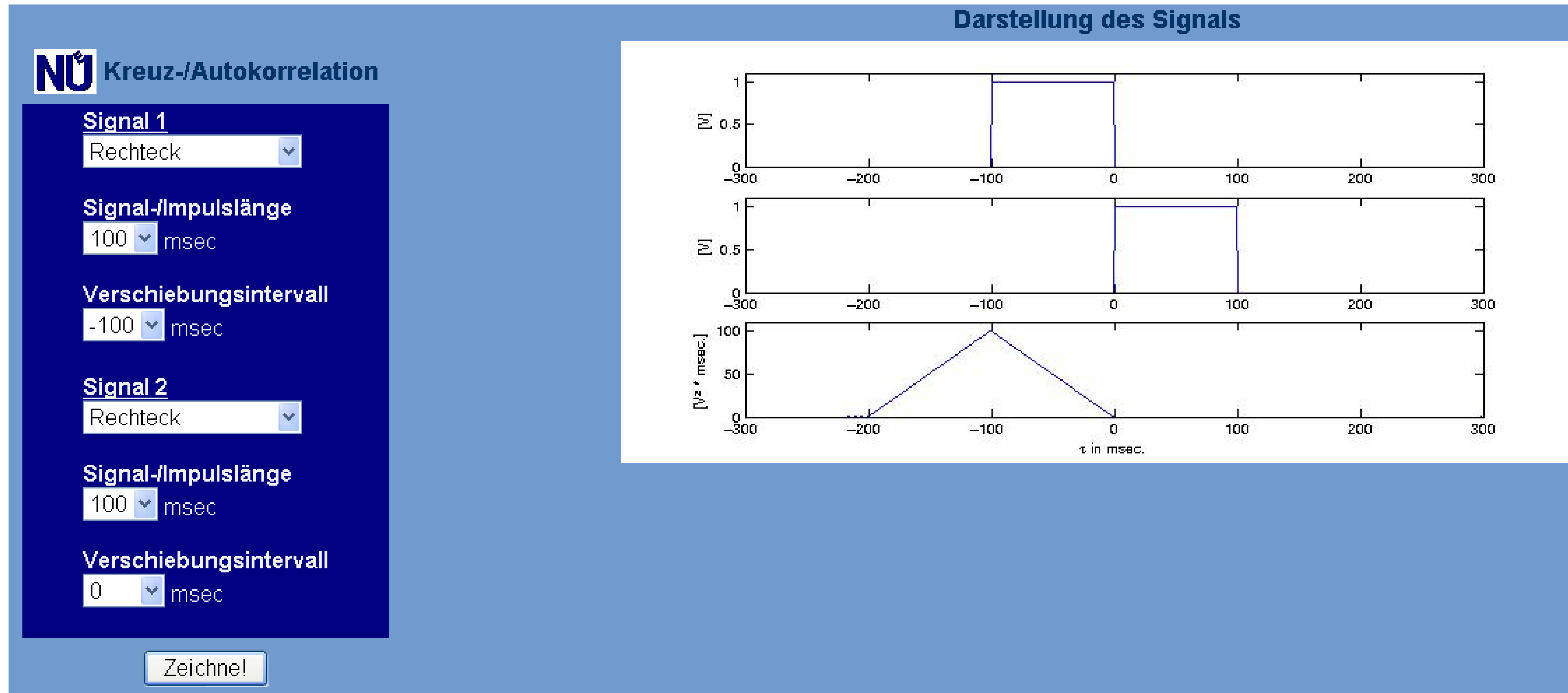
Cross-correlation Function

- CCF, measures similarity between 2 signals

$$r_{xy}(k) = \sum_{n=-\infty}^{\infty} x(n) \cdot y(n+k)$$

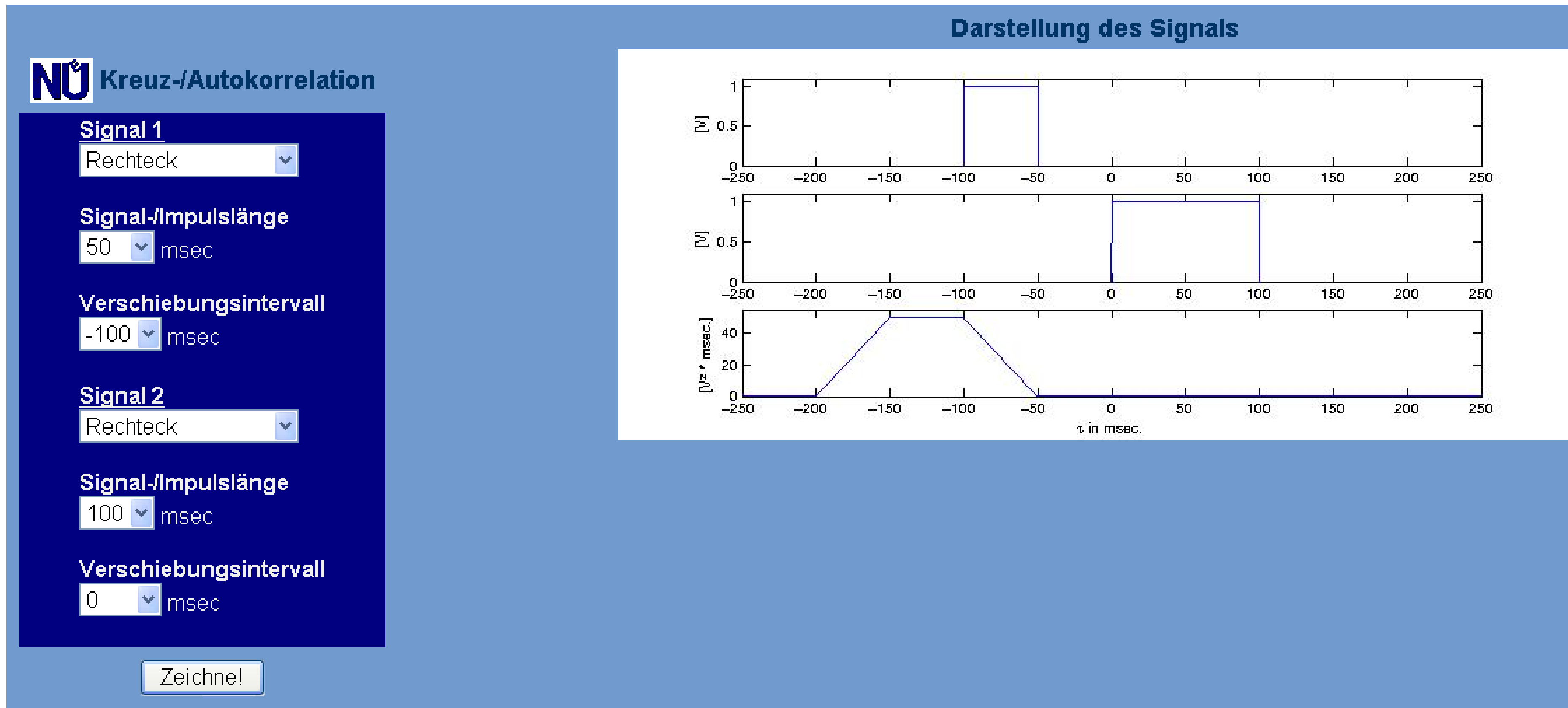
CCF Examples

- CCF of 2 shifted rectangular functions



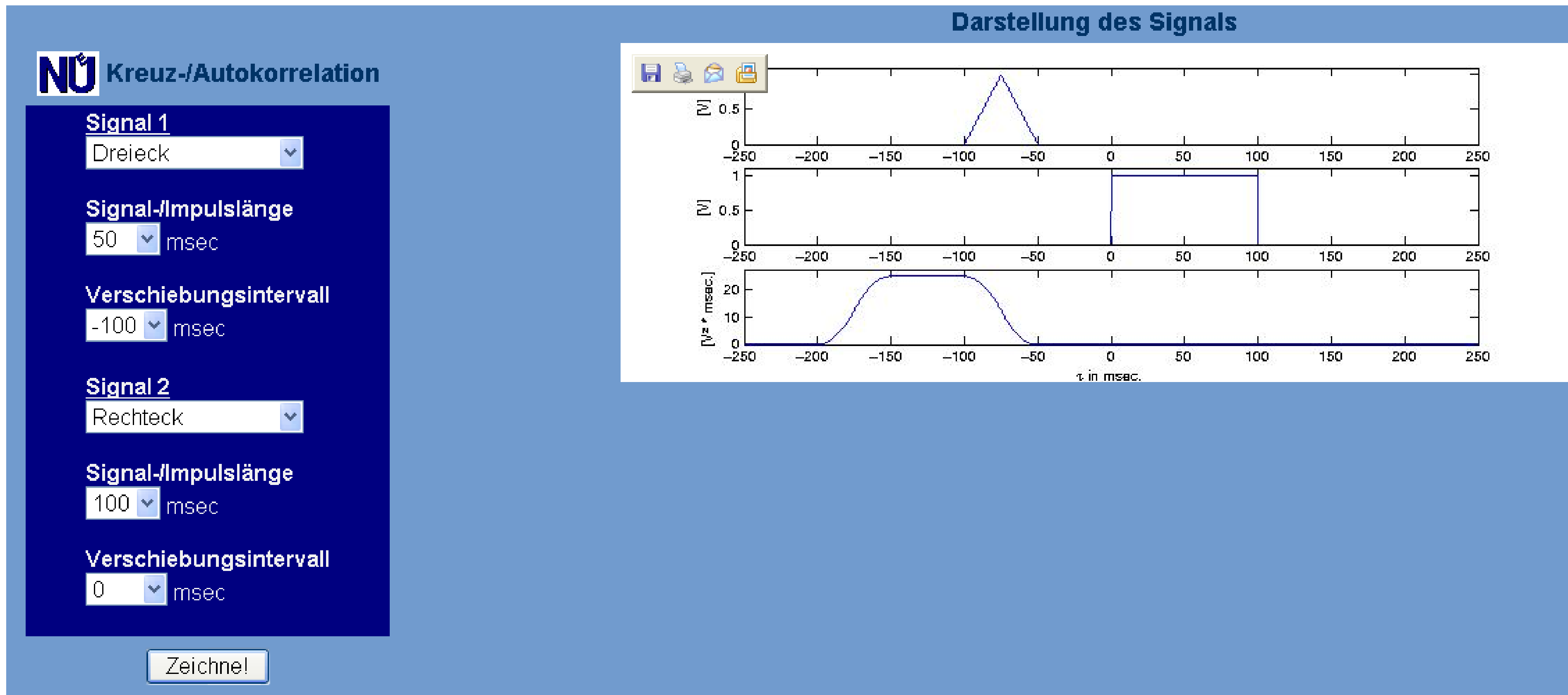
CCF Examples

- CCF of 2 shifted rectangular functions of different length



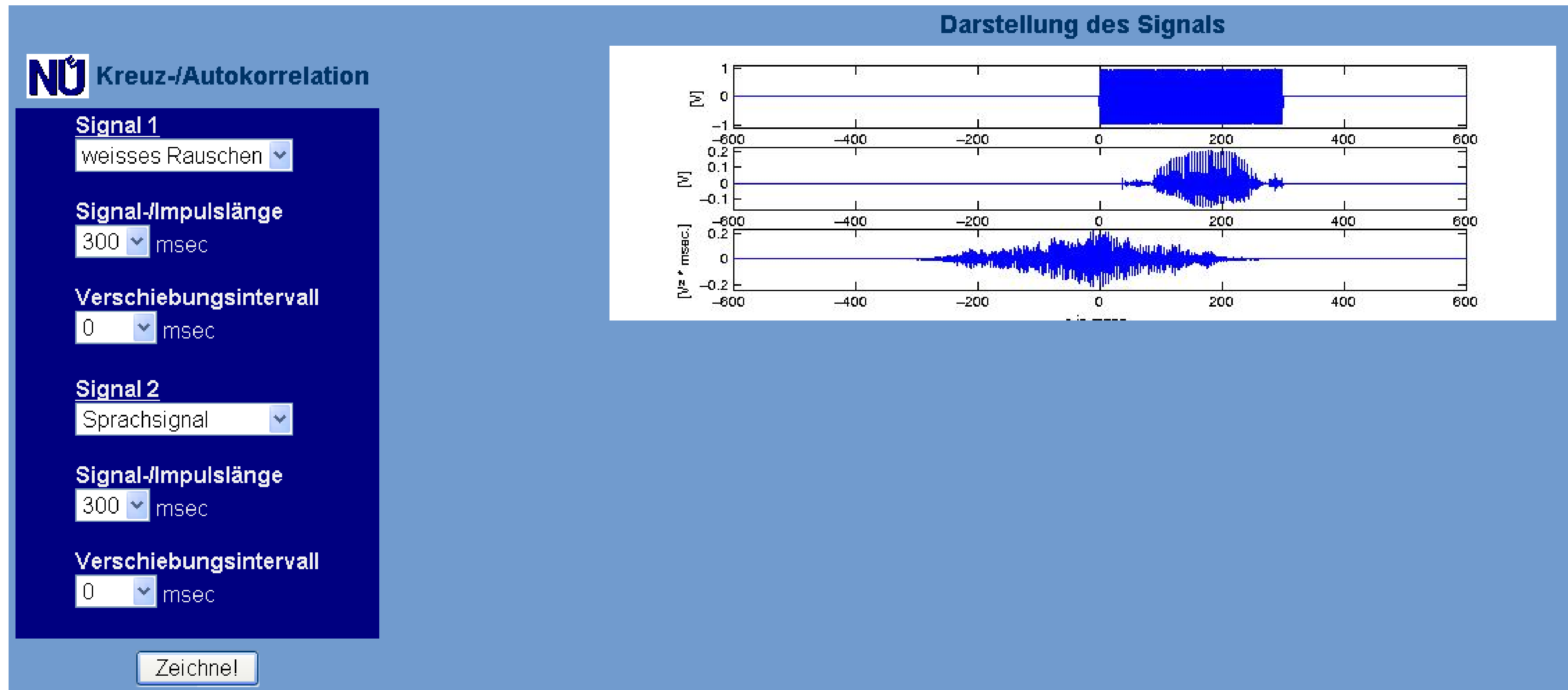
CCF Examples

- CCF of shifted rectangular and triangular functions of different length



CCF Examples

- CCF of a speech and a noise signal



1D (Cross-) Correlation

- The correlation between the sequences x and y measures their similarity.
- The definition of correlation is similar to the definition of convolution:

$$\text{corr}(x, y)_k = \sum_{i=-\infty}^{\infty} x_i \cdot y_{k+i}$$

- Often the normalized cross-correlation is used:

$$\text{ncc}(x, y)_k = \frac{\sum_{i=-\infty}^{\infty} x_i \cdot y_{k+i}}{\sqrt{\left(\sum_{i=-\infty}^{\infty} x_i^2\right) \cdot \left(\sum_{i=-\infty}^{\infty} y_{k+i}^2\right)}}$$

Properties of the normalized cross-correlation:

- $\text{ncc}(x, x)_k$ is called autocorrelation
- $-1 \leq \text{ncc}(x, y)_k \leq 1$
- if x shifted by k is equal to y :
$$\text{ncc}(x, y)_k = 1$$
- if x shifted by k is in phase opposition to y :
$$\text{ncc}(x, y)_k = -1$$
- if the signals x and y are uncorrelated:
$$\text{ncc}(x, y)_k = 0$$

for every k

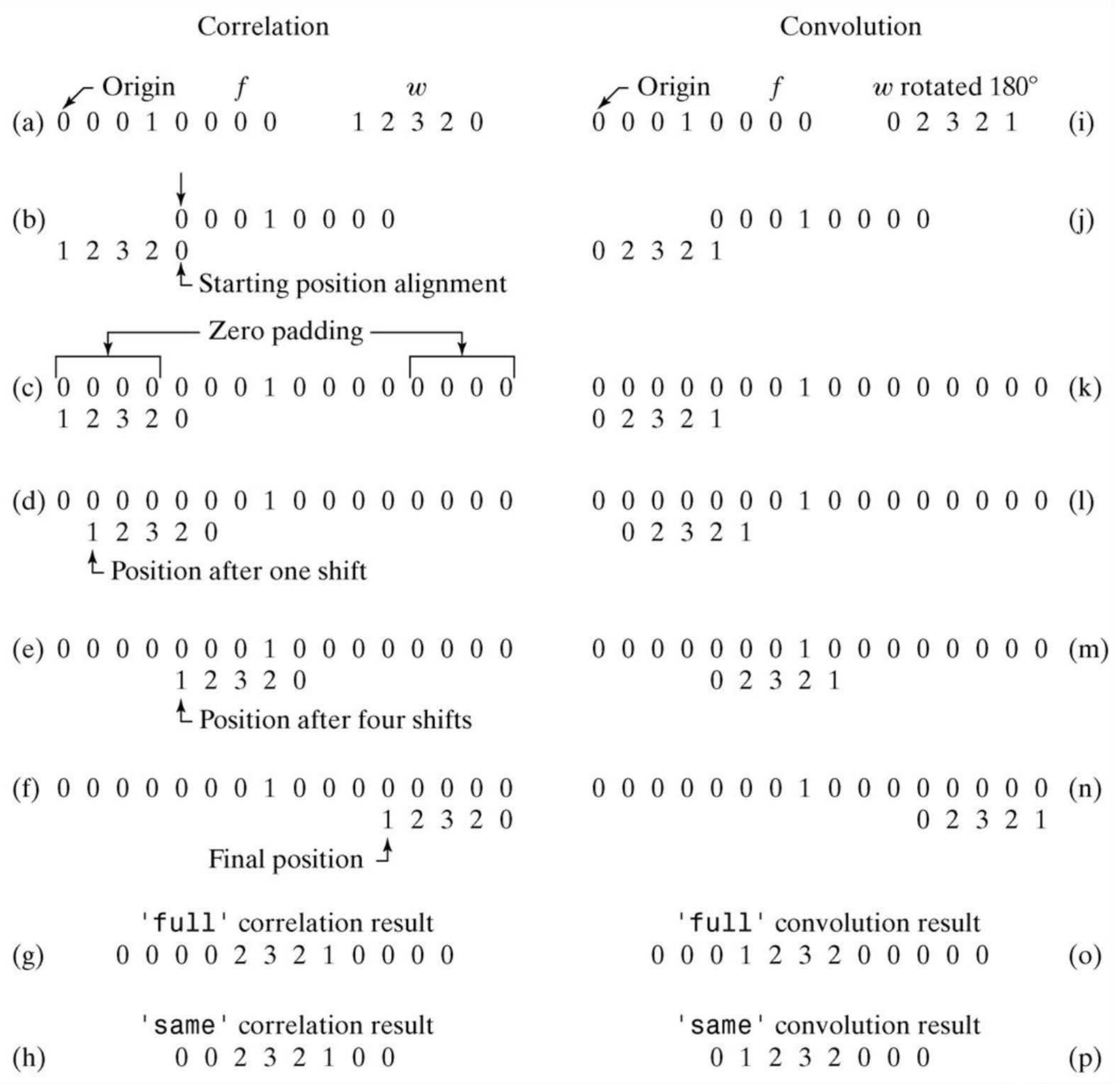
1D (Cross-) Correlation

Image:
 $f = (0, 0, 0, 1, 0, 0, 0, 0)$

Filter:
 $w = (1, 2, 3, 2, 0)$

Correlation:
Shift kernel left
until overlap is 1 pixel.

Convolution:
Reflect kernel at first pixel.



1D (Cross-) Correlation

Application:

Pattern recognition in 1D or 2D

Use filter kernel similar to pattern

Exercise:

Find pattern (3, 7, 5)

in "image" (3, 2, 4, 1, 3, 8, 4, 0, 3, 2, 3, 2)

Solution: position 4

2D (Cross-) Correlation

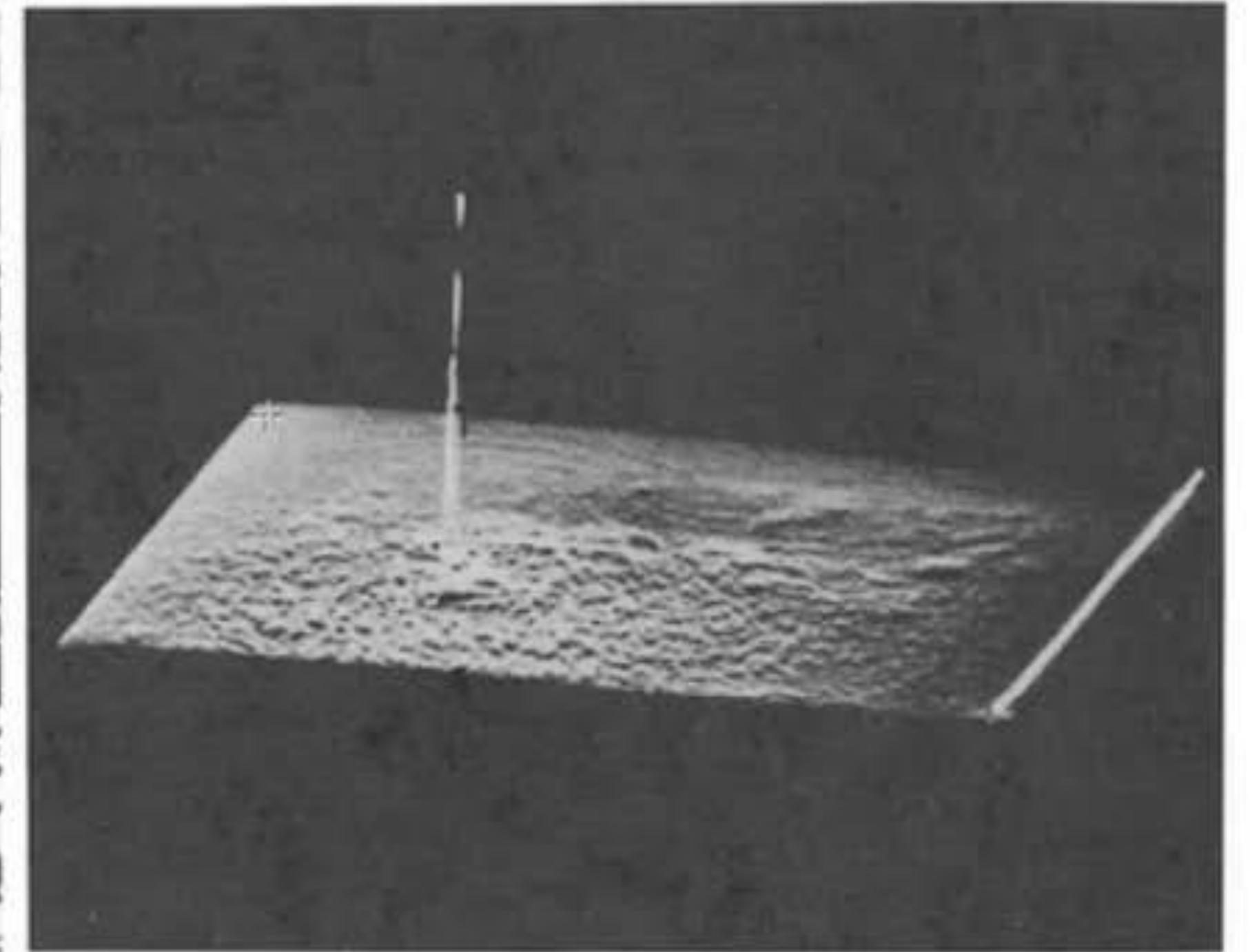
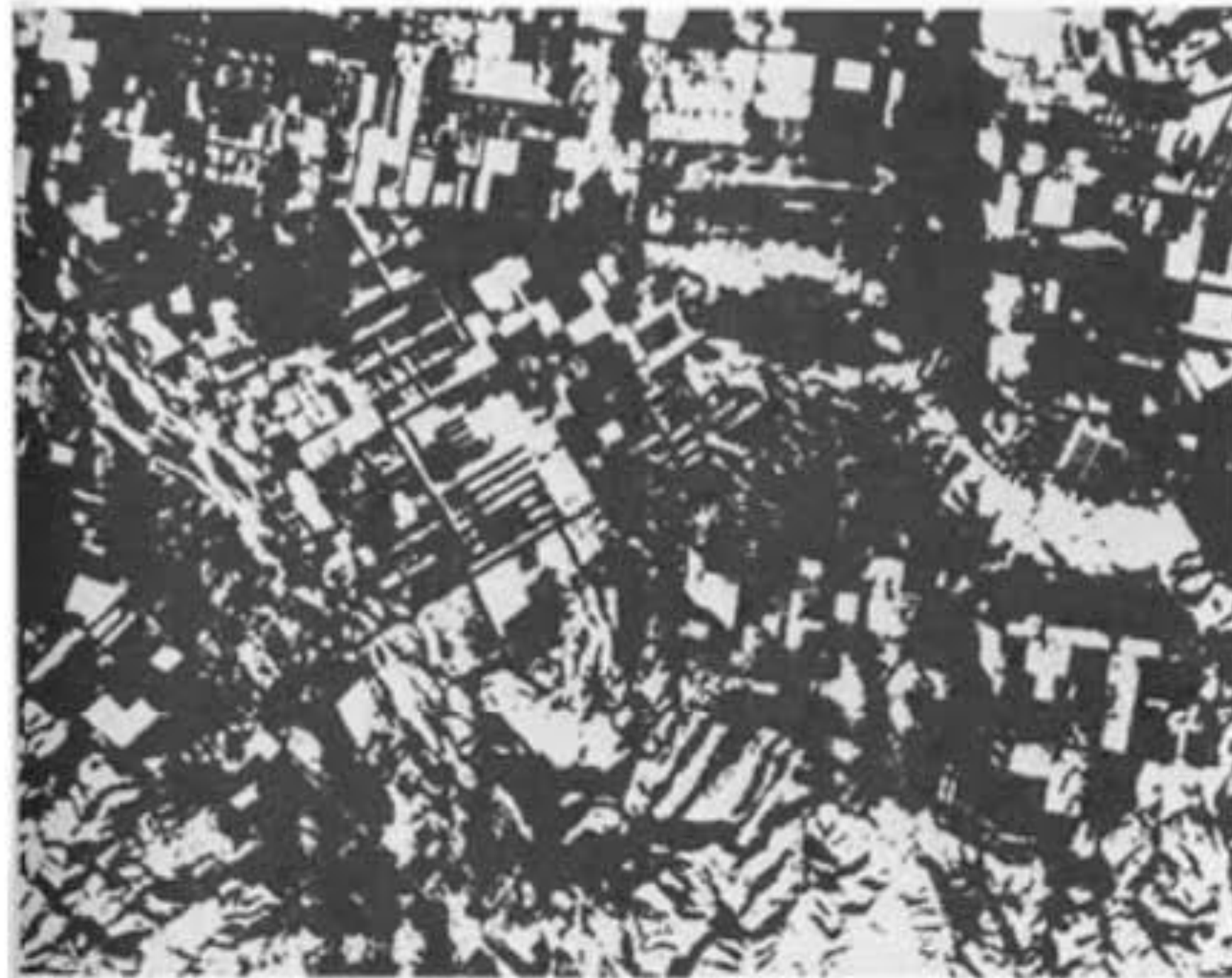
$$h = \text{ncc}(f, g)$$

$$h(u, v) = \frac{\sum_{(x,y) \in R} f(x, y) \cdot g(x + u, y + v)}{\sqrt{\left(\sum_{(x,y) \in R} (f(x, y))^2 \right) \cdot \left(\sum_{(x,y) \in R} (g(x + u, y + v))^2 \right)}}$$

- R: range of image f (pattern)
- Properties: see 1D cross-correlation

2D (Cross-) Correlation

Cross-
correlation



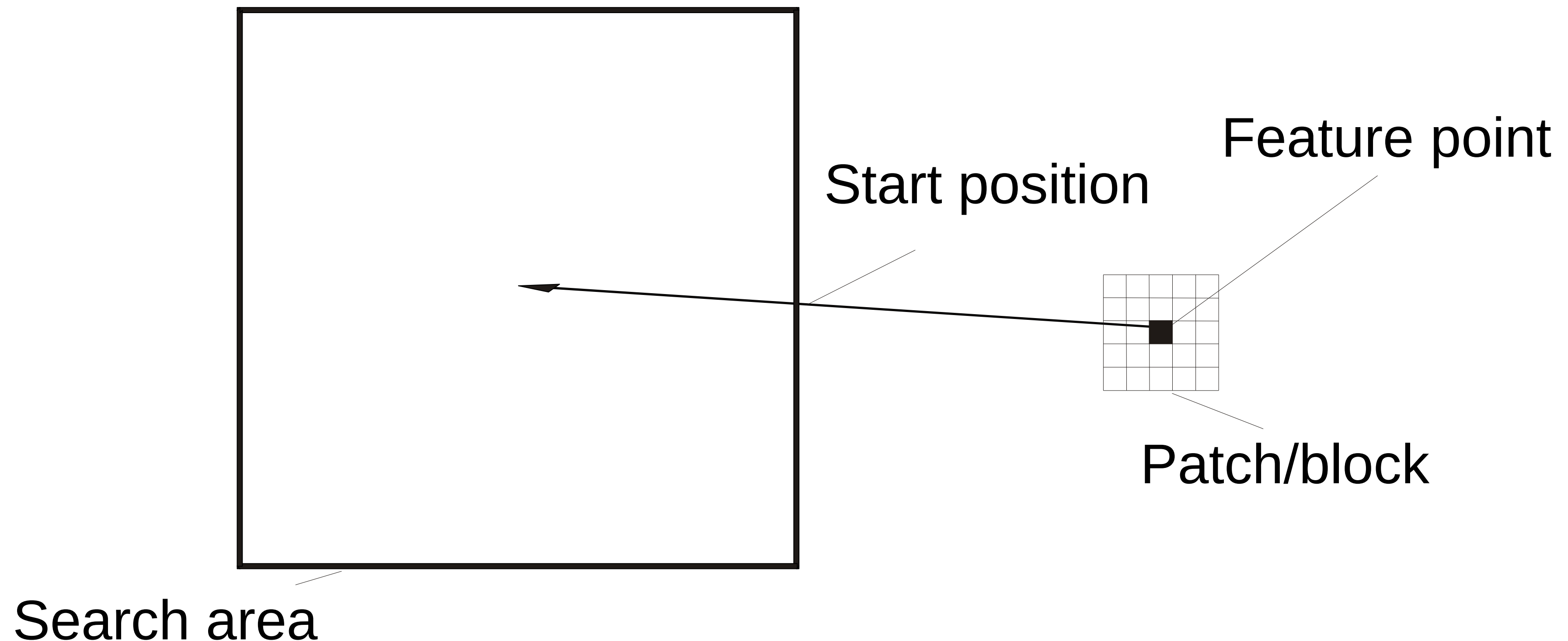
Hecht, Optics



Feature matching

- Find optimum displacement vector
- For a patch or block around a feature point
- Based on a similarity measure

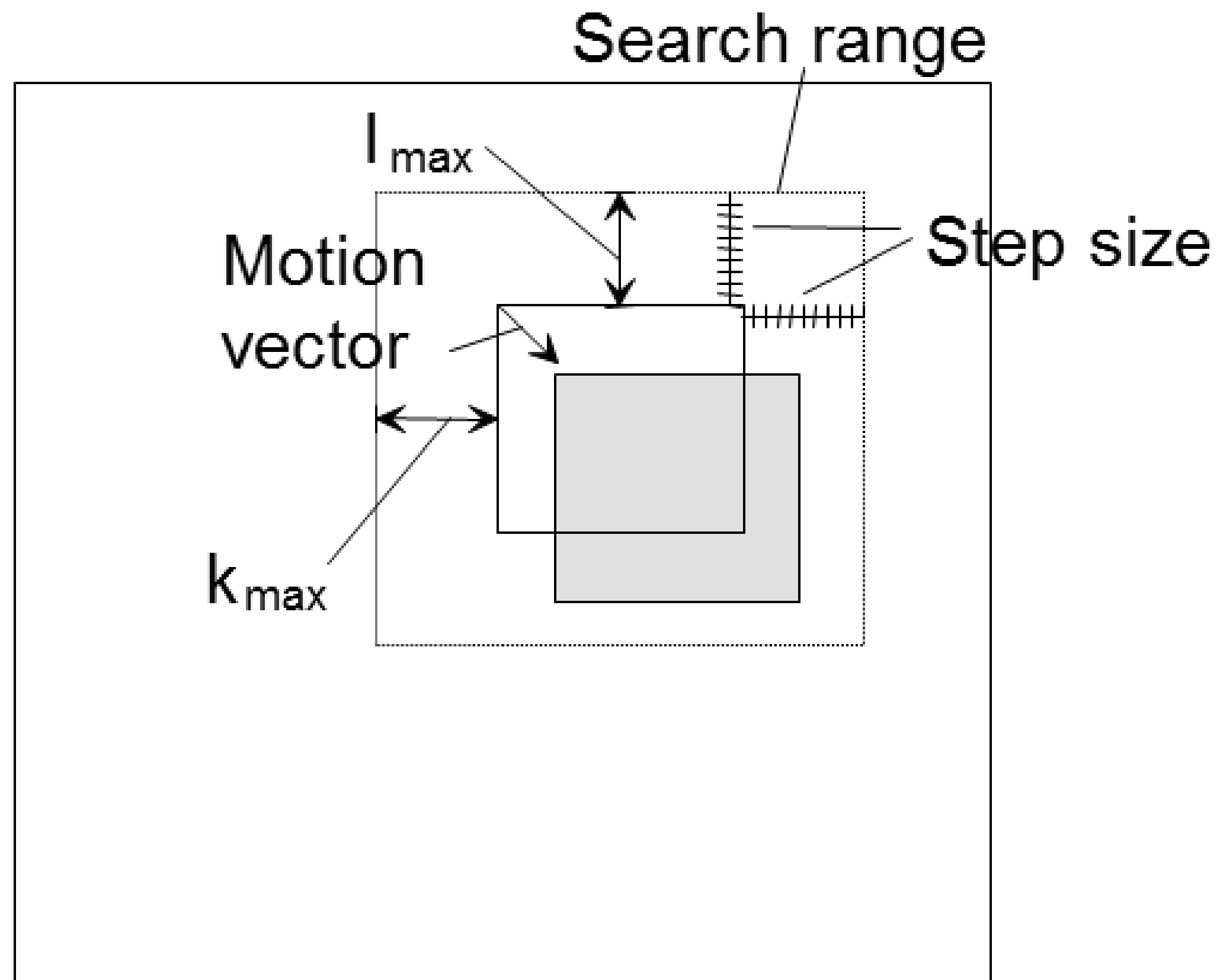
$$E_{\text{WSSD}}(\mathbf{u}) = \sum_i w(\mathbf{x}_i) [I_1(\mathbf{x}_i + \mathbf{u}) - I_0(\mathbf{x}_i)]^2$$



Feature matching

- Find optimum displacement vector
- For a patch or block around a feature point
- Based on a similarity measure

$$E_{\text{WSSD}}(\mathbf{u}) = \sum_i w(\mathbf{x}_i) [I_1(\mathbf{x}_i + \mathbf{u}) - I_0(\mathbf{x}_i)]^2$$



2D (Cross-) Correlation vs Convolution

5x5 image f , 3x3 kernel w

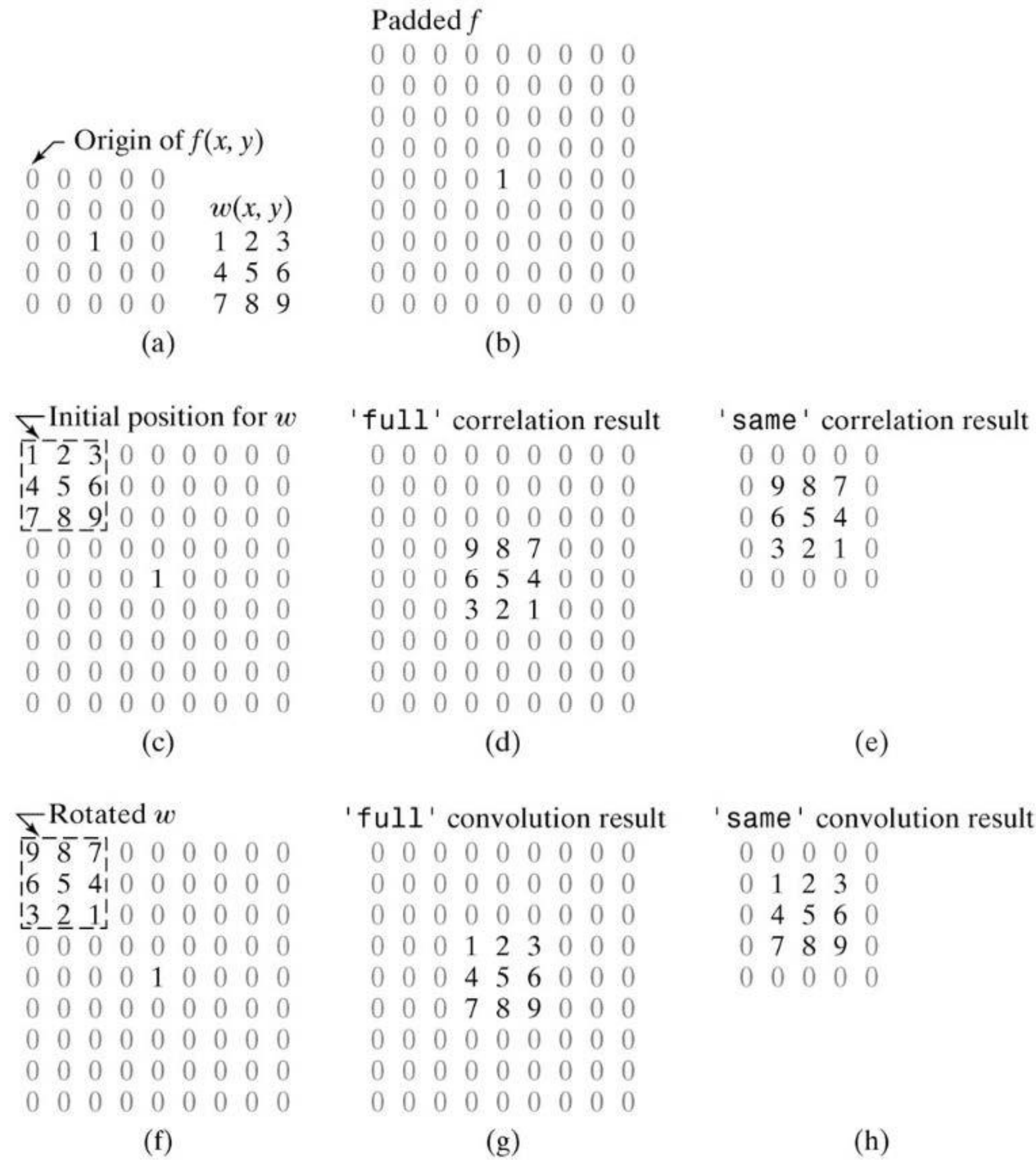
- correlation (c, d, e)
- convolution (f, g, h)

'same' (e, h)

- result size is equal to size of f

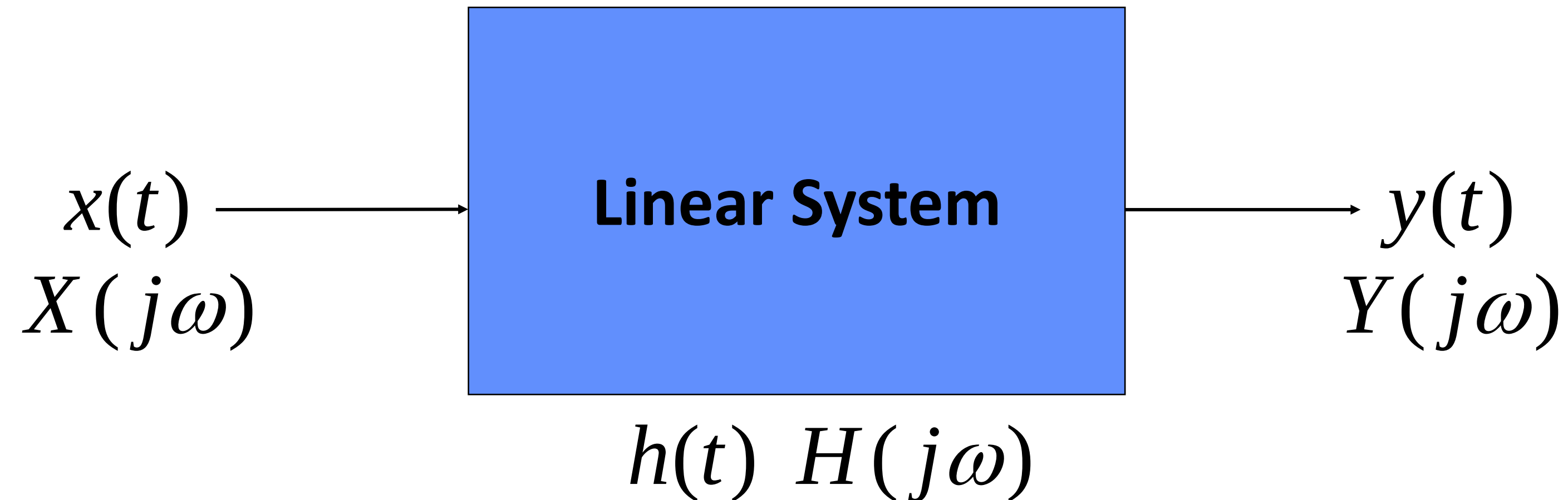
'full' (d, h)

- f has size $m \times n$
- w has size $p \times q$
- result has size $(n + p + 1) \times (m + q + 1)$



Linear Systems

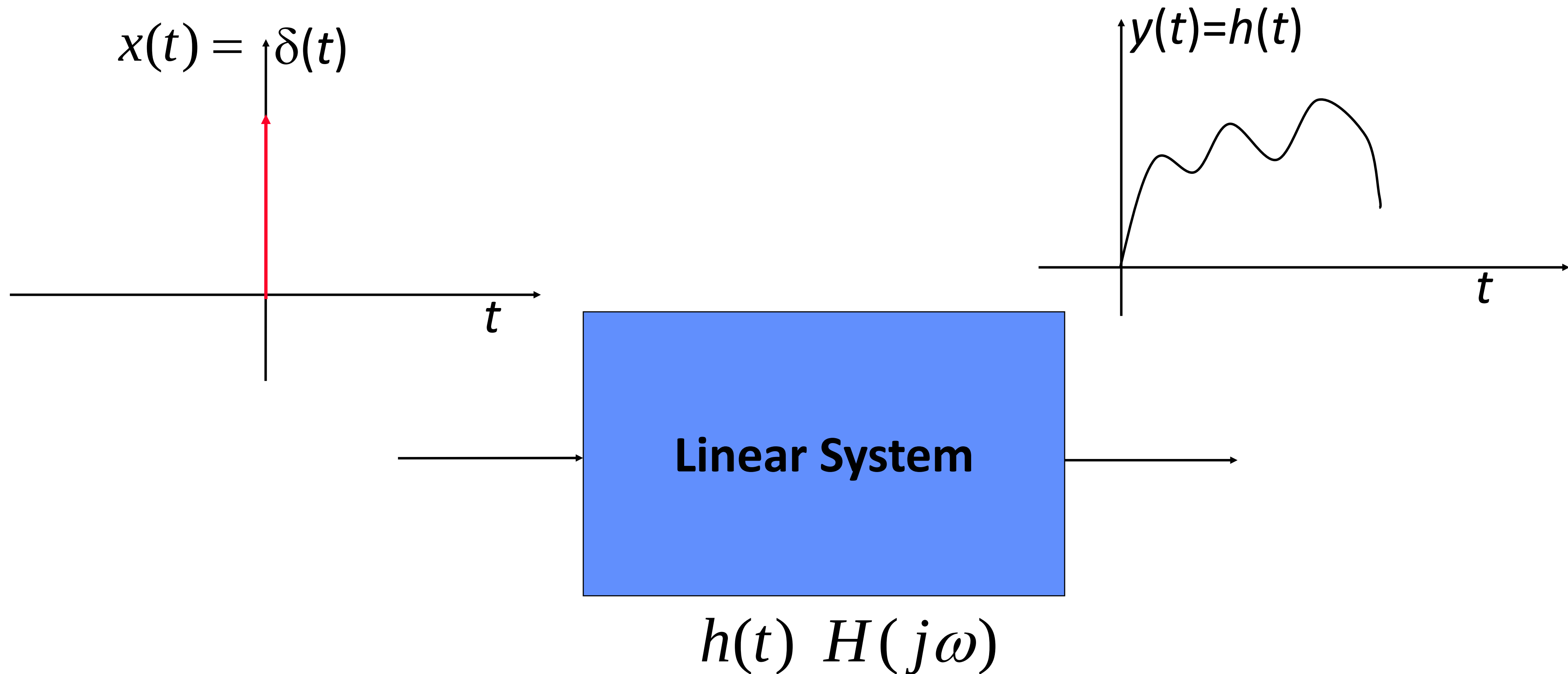
- A linear systems transforms an input signal into an output signal



- A linear system is defined by the transfer function $H(j\omega)$ and the impulse response $h(t)$

Linear Systems

- The impulse response $h(t)$ is the output of a linear system to a delta function $\delta(t)$ as input



Linear Systems

- Every signal can be represented as an infinitely dense sequence of weighted impulses
- The output of a linear system can be calculated as an infinitely dense superposition of weighted impulse responses
- Output is convolution of input and impulse response

$$y(t) = \int_{-\infty}^{\infty} x(\tau) \cdot h(t - \tau) d\tau = x(t) * h(t)$$

Linear Systems

- The transfer function of a linear system is the Fourier transform of the impulse response

$$H(j\omega) = \int_{-\infty}^{\infty} h(t) \cdot e^{-j\omega t} dt$$

- Vice versa

$$h(t) = \frac{1}{2\pi} \int_{-\infty}^{\infty} H(j\omega) \cdot e^{j\omega t} d\omega$$

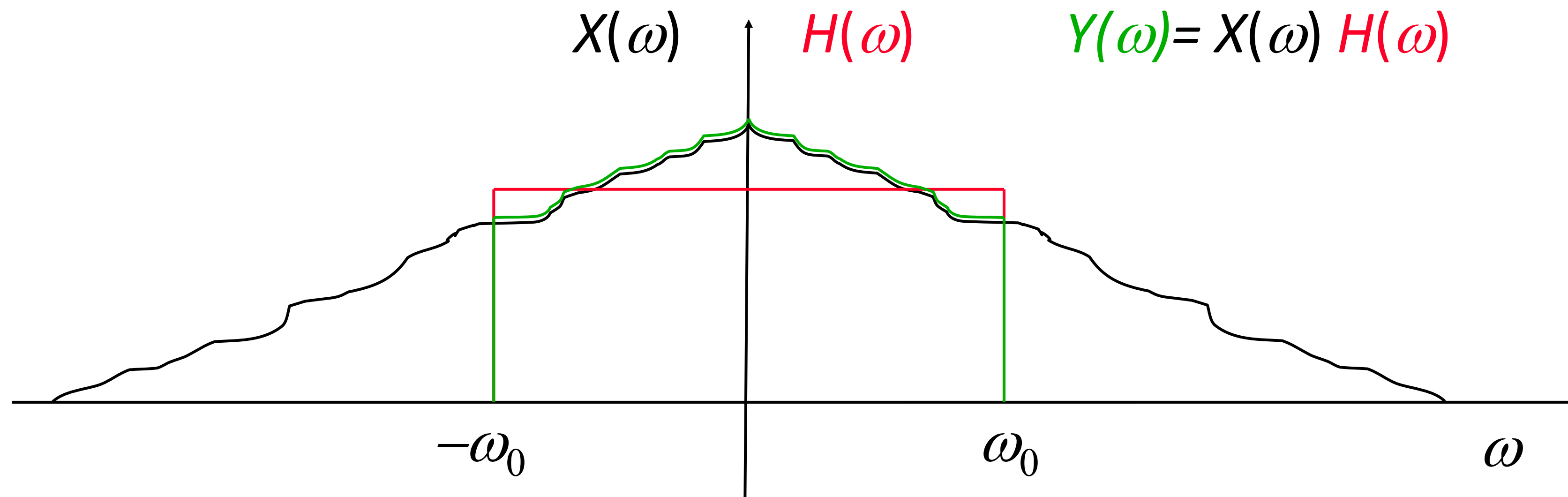
Linear Systems

- Convolution theorem of Fourier transform:
- Convolution in time domain results in multiplication in frequency domain
- Output spectrum is input spectrum multiplied by transfer function

$$Y(j\omega) = X(j\omega) \cdot H(j\omega)$$

Linear Systems

- Example: ideal low pass filter



Discrete Linear Systems

- 2D low pass



Discrete Linear Systems

- 2D high pass



Linear Filters

- Linear combination of pixels in a neighborhood
- images: x and y
- filter: f
- linear filter: $f(x + y) = f(x) + f(y)$
- $f(x + y)$: add images first, then apply filter
- $f(x) + f(y)$: apply filters first, add results
- $f(x + y) \neq f(x) + f(y)$: nonlinear filter

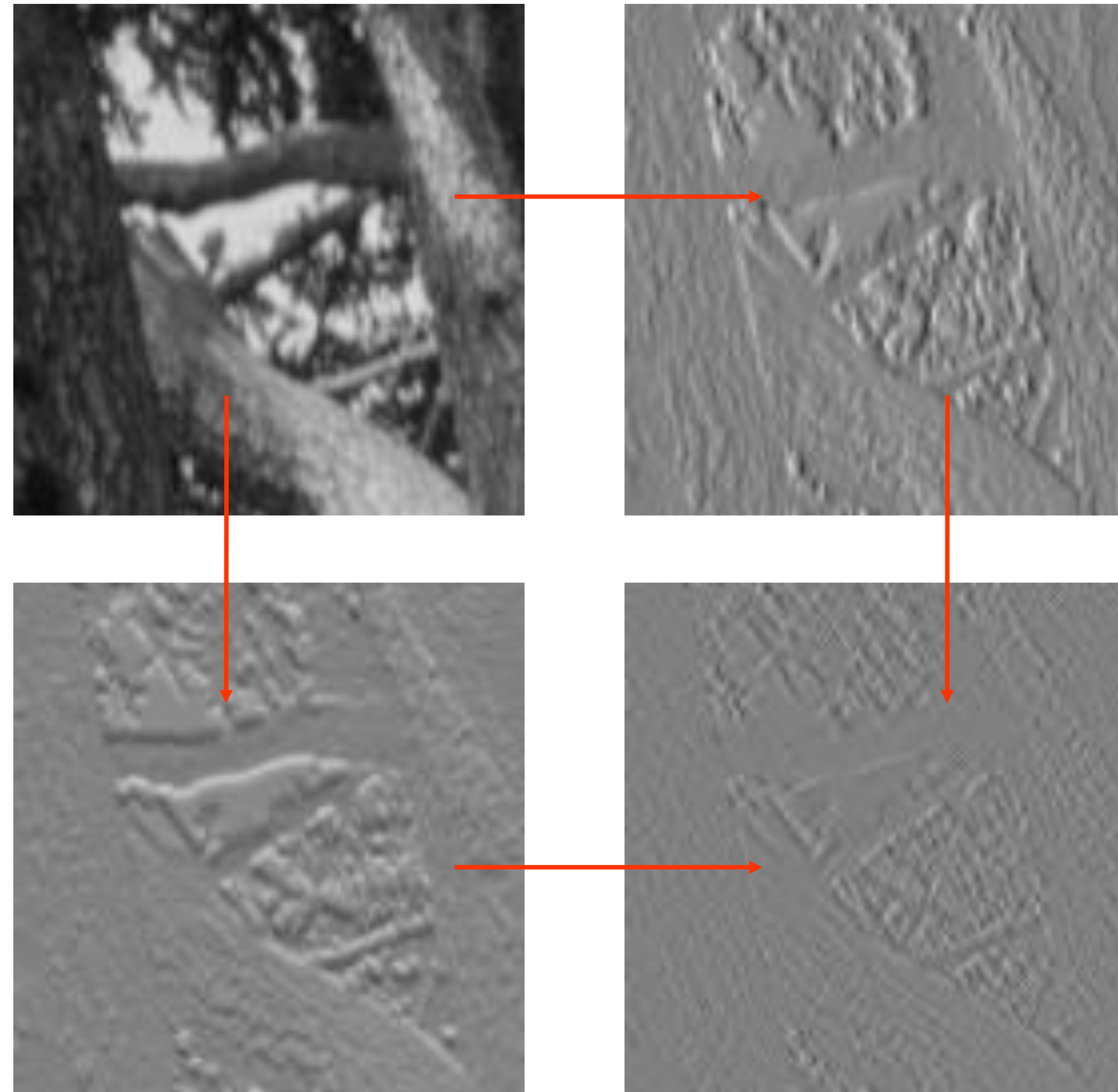
Low pass filter

- make image smoother (Hamilton effect)
- blur edges
- reduce noise and details
- leave homogenous regions unchanged

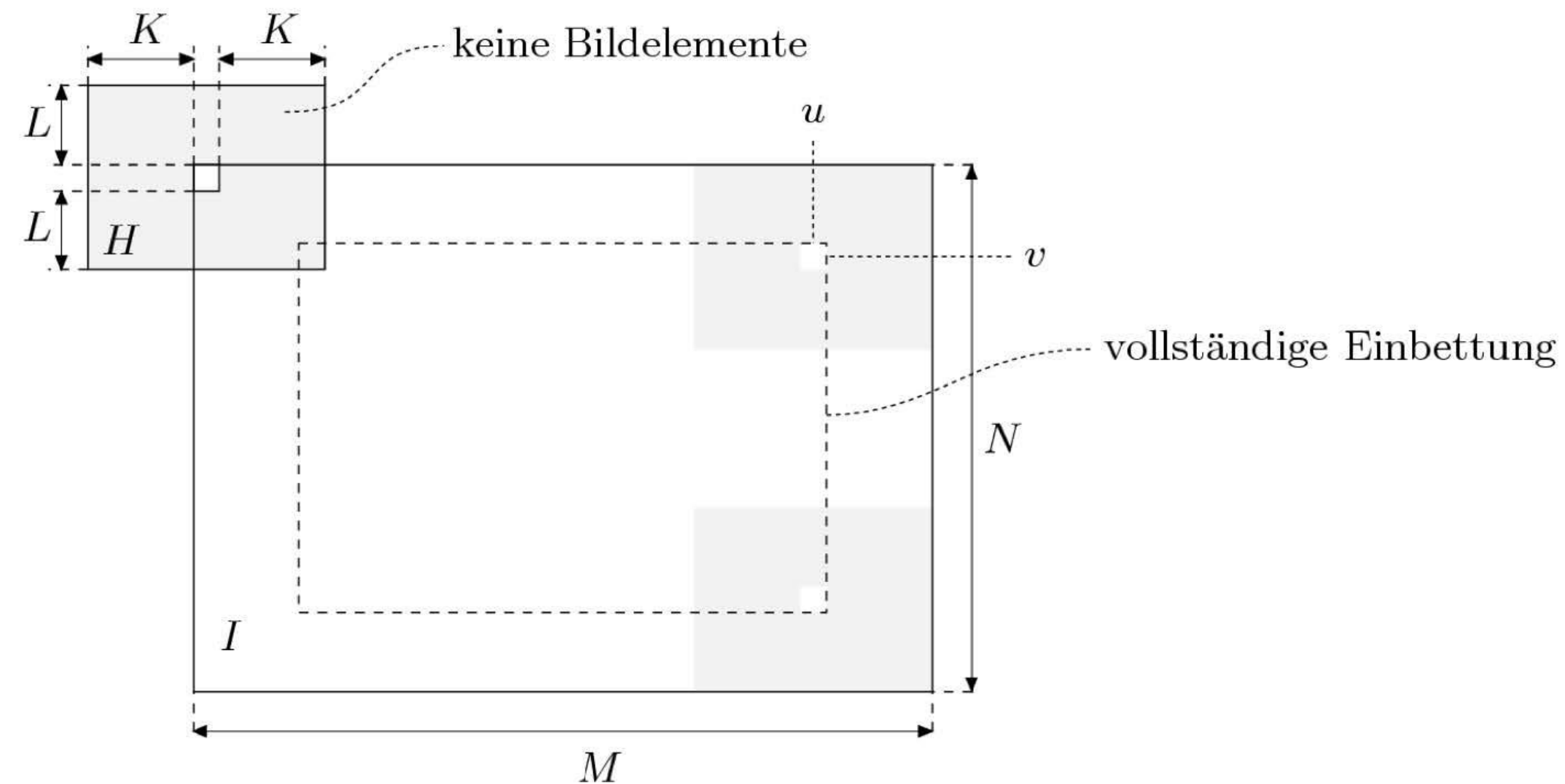
High pass filter

- sharpen image
- amplify edges and details
- eliminate homogenous regions

Prediction of Image



How to Handle Border Pixels?



Possible solutions

- no filtering in the near of borders
- set arbitrary value outside image (gray or zero padding)
- use values of border pixels outside image (last value)
- periodic extension
- reflect image at border (symmetric extension)
- extrapolation across the border
- construct special filter masks

Examples of Linear Filters

Low pass filters

- mean filter
(arithmetic mean of neighborhood)
- gaussian filter
approximation of normal distribution
less blur compared to mean filter

$$\frac{1}{9} \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$$

$$\frac{1}{16} \begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{pmatrix}$$

High pass filters (1)

- Prewitt operator

$$P_x = \begin{pmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{pmatrix} \quad P_y = \begin{pmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{pmatrix}$$

- Sobel operator

$$S_x = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix} \quad S_y = \begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{pmatrix}$$

- Sobel operator (diagonal)

$$S_x = \begin{pmatrix} 0 & -1 & -2 \\ 1 & 0 & -1 \\ 2 & 1 & 0 \end{pmatrix} \quad S_y = \begin{pmatrix} -2 & -1 & 0 \\ -1 & 0 & 1 \\ 0 & 1 & 2 \end{pmatrix}$$

High pass filters (2)

- Laplace
(pixels with highest "curvature")
- Compass filters
applying 45° rotations to P_x and S_x
gives 8 variations in each case.

$$L_1 = \begin{pmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{pmatrix} \quad L_2 = \begin{pmatrix} 0 & -1 & -1 \\ -1 & 6 & -1 \\ -1 & -1 & 0 \end{pmatrix}$$

$$L_3 = \begin{pmatrix} 1 & -2 & 1 \\ -2 & 4 & -2 \\ 1 & -2 & 1 \end{pmatrix} \quad L_4 = \begin{pmatrix} -1 & -2 & -1 \\ -2 & 12 & -2 \\ -1 & -2 & -1 \end{pmatrix}$$